



UNIVERSITE DE FIANARANTSOA

ECOLE NATIONALE D'INFORMATIQUE

**MEMOIRE DE FIN D'ETUDES EN VUE DE L'OBTENTION DU DIPLOME
DE MASTER PROFESSIONNEL**

Mention : Informatique

Parcours : Informatique Général

Intitulée0 :

**MODERISATION TECHNOLOGIQUE DU
SERVICE MEDICALE INTERENTREPRISE
FUNTOA VIA UNE APPROCHE CI/CD**

Présentée le **03 Décembre 2025**

Par Monsieur **RAZAFIMAHARO** Mamy Tiandrainy *Lolito*

Membres Du Jury :

- **Président** : Monsieur **RANDRIANOMENJANAHARY** Lala Ferdinand, Maître de Conférences
- **Examineur** : Monsieur **RAZAFINDRAMONJA** Clément Aubert, Assistant d'Enseignement Supérieur et de Recherche
- **Rapporteur** : Monsieur **RAFAMANTANATSOA** Fontaine, Professeur

Année Universitaire : 2024-2025

CIRRICULUM VITAE

Nom : RAZAFIMAHARO

Prénoms : Mamy Tiandrainy *Lolito*

Née le : 02 décembre 2001 à Bekitro/Bekily

Adresse : Lot 182bis parcelle 12/21/22 Toamasina 501

Contact : +261 34 75 075 76

Email : lolitorazafimaharo@gmail.com



FORMATIONS ET DIPLÔMES OBTENUS

Ecole Nationale d'Informatique (E.N.I)

2023-2024 : deuxième année de master en Informatique Générale

2023-2024 : première année de master en Informatique Générale

2022-2023 : troisième année de licence en Informatique Générale et obtention de diplôme licence professionnelle

Ecole Sacré-Cœur Anosy Taolagnaro (ESCATO)

2018-2019 : classe de terminale série D et obtention de diplôme baccalauréat

EXPERIENCES PROFESSIONNELLES :

Juillet 2025 – en ce jour : Responsable informatique au sein de la société FUNTOA SMIE

Taches réalisées :

- ❖ Projet du stage : « Modernisation technologique du Service Médical Interentreprises via une approche CI/CD »
- ❖ Reconfiguration de système réseau et prise en contrôle le système réseau

Septembre-novembre 2023 : Stagiaire au sein de l'entreprise TAKALOU à Taolagnaro/Fort Dauphin.

Tâches réalisées :

- ❖ Les multiples interventions englobent divers domaines tels que la maintenance du réseau, le support matériel et logiciel, ainsi que d'autres aspects liés aux technologies de l'information.

- ❖ Projet du stage : « Mise en place d'un serveur Samba AD, partage de fichiers et déploiement Docker PHP/Apache MySQL. »

Septembre-décembre 2022 : Stagiaire binôme au sein de l'entreprise Lazan'ny Betsileo à Isaha Fianarantsoa.

Tâche réalisée : Projet du stage : « Conception et réalisation de gestion des commandes »

Mai-Août 2022 : Gérant cyber et multi service « Henri Sardou » à Ambatotokana Soanierana Fianarantsoa.

Tâches réalisées :

- ❖ Tous les services du multiservice et des divers service informatique (installation système Windows/linux, réparation...)
- ❖ *Installation système électrique*

2020-2025 : ECOLE NATIONALE D'INFORMATIQUE Réalisation des projets d'études

CONNAISSANCES INFORMATIQUES

- **Administration en Système et Réseaux**
 - **Système d'exploitation** : Windows, Linux
 - **Langage de programmation** : JavaScript, Python
 - **Framework** : Bootstrap
 - **Serveur web** : Nginx
 - **Base de données** : PostgreSQL, MySQL, MongoDB
 - **Intégration Continue/Déploiement Continu** : GitHub Actions
 - **Gestion des Conteneurs** : Docker, Docker Compose
-

CONNAISSANCES LINGUISTIQUES

	Comprendre à l'audition	Lire	Ecrire et rédiger	Parler et communiquer
Français	Bien	Bien	Bien	Bien
Anglais	Assez-bien	Bien	Bien	Assez bien

CENTRES D'INTERÊT :

- **SPORTS** : football, natation
- **Loisirs** : écoutez de la musique, jouer au jeu vidéo, jouer aux échecs

VIE ASSOCIATIVE :

- ✚ Membre de l'association **FI.MPL.ZA** (Fikambanan'ny Mpianatry Zanak'Anosy)

SOMMAIRE

CIRRICULUM VITAE	i
SOMMAIRE	iv
REMERCIEMENTS	vi
LISTE DES FIGURES	vii
LISTE DES TABLEAUX	viii
LISTE DES ABREVIATIONS.....	ix
INTRODUCTION GENERALE.....	1
PARTIE I : PRESENTATIONS	2
Chapitre 1: Présentation de l'Ecole Nationale d'Informatique	3
Chapitre 2: Présentation de SMIE FUNTOA.....	13
Chapitre 3: Présentation et cadrage du projet.....	20
PARTIE II : ANALYSE ET CHOIX DES OUTILS	30
Chapitre 4: Analyse préalable et conception fonctionnelle	31
Chapitre 5: Revue de littérature	40
Chapitre 6: Choix des outils	44
PARTIE III : MISE EN ŒUVRE	57
Chapitre 7: Réalisation du projet.....	58
Chapitre 8: Test et validation.....	84
CONCLUSION	90
BIBLIOGRAPHIE	xi
WEBOGRAPHIE.....	xii
GLOSSAIRE	xiii
ANNEXE	xiv
TABLE DES MATIERES	xv

RESUME.....xxi

ABSTRACTxxi

REMERCIEMENTS

Mes premiers remerciements vont tout d'abord à Dieu Tout-Puissant, qui m'a donné la force, le courage et la persévérance nécessaires pour mener à bien ce projet.

Je tiens également à exprimer ma profonde gratitude aux personnes suivantes pour leurs contributions essentielles à la réalisation de ce mémoire :

- **Monsieur HAJALALAINA Aimé Richard**, Docteur HDR, Président de l'Université de Fianarantsoa, pour les efforts qu'il a déployés afin d'assurer le bon déroulement de l'année universitaire.
- **Monsieur MAHATODY Thomas**, Docteur HDR, Directeur de l'École Nationale d'Informatique, qui nous a permis d'accomplir nos années d'études au sein de la formation en Informatique Générale.
- **Monsieur RASENDRALAHY Yvon Joël**, Directeur exécutif de la société FUNTOA SMIE, pour m'avoir accordé l'opportunité d'effectuer mon stage au sein de son entreprise.
- **Monsieur RAFAMANTANATSOA Fontaine**, Professeur, pour avoir accepté d'être mon rapporteur.
- **Monsieur RANDRIANOMENJANAHARY Lala Ferdinand**, Maître de Conférences, de bien avoir accepté de président de Jury de ce mémoire.
- **Monsieur RAZAFINDRAMONJA Clément Aubert**, Assistant d'Enseignement Supérieur et de Recherche, Examineur qui a accepté d'évaluer ce travail de mémoire.

Je tiens également à témoigner toute ma reconnaissance à l'ensemble du corps enseignant de l'ENI ainsi qu'aux intervenants responsables de la formation au sein du département Informatique.

J'adresse aussi mes sincères remerciements à ma famille et à mes amis pour leur soutien, leurs encouragements et leur présence à chaque étape de l'élaboration de ce travail.

Enfin, pour toutes les personnes qui ont contribué de près ou de loin à l'élaboration de ce mémoire. Je ne pourrais pas tous les énumérer, qu'ils trouvent ici également mes sincères remerciements.

LISTE DES FIGURES

Figure 1: Organigramme de l'Ecole Nationale d'Informatique	6
Figure 2: Architecture des études correspondant au système LMD.....	7
Figure 3 : Diagramme de GANTT	29
Figure 4 : Diagramme de cas d'utilisation global	39
Figure 5 : Cycle de vie DevOps	43
Figure 6 : Illustration du pipeline CI/CD Github action, docker	53
Figure 7 : Offre VPS OVHCloud	59
Figure 8 : Tableau de bord OVHCloud	60
Figure 9 : Bitwise SSH Client non connecté.....	61
Figure 10 : Connexion établie	62
Figure 11 : Terminal sur le serveur.....	63
Figure 12 : Téléchargement et installation Git.....	64
Figure 13 : Téléchargement et installation VSCode.....	67
Figure 14 : Téléchargement et installation docker desktop.....	68
Figure 15 : Docker desktop image local.....	68
Figure 16 : Dockerfile du backend	71
Figure 17 : Dockerfile du frontend.....	72
Figure 18 : Job validate code.....	81
Figure 19 : Job build and push docker image.....	82
Figure 20 : Job deploy to VPS.....	83
Figure 21 : Action Push vers Central Hub (Github).....	84
Figure 22 : Phase de build du pipeline en cours d'exécution.....	85
Figure 23 : Phase de build du pipeline en cours d'exécution.....	85
Figure 24 : Succès du pipeline	86
Figure 25 : Login.....	87
Figure 26 : État initial du code déployé	88
Figure 27 : Améliorations et tests après modification.....	88
Figure 28 : Validation des améliorations mises en production sur le nom de domaine	89

LISTE DES TABLEAUX

Tableau 1: Organisation du système de formation pédagogique de l'Ecole	6
Tableau 2: Mention et parcours au sein de l'ENI.....	7
Tableau 3: Liste des formations existantes à l'ENI.....	8
Tableau 4: Débouchés éventuels des jeunes diplômés	10
Tableau 5 : Configuration des moyens matériels pour la réalisation du projet	24
Tableau 6 : Chronogramme des activités du stage	27
Tableau 7 : Configuration des ordinateurs existants	32
Tableau 8 : Inventaire des imprimantes.....	32
Tableau 9 : Comparaison Docker et Podman	46
Tableau 10 : Comparaison GHCR et Docker Hub	49
Tableau 11 : Comparaison Github Actions avec Jenkins et Gitlab CI/CD.....	51

LISTE DES ABREVIATIONS

API : Application Programming Interface

CI : Continuous Integration (Intégration Continue)

CD : Continuous Deployment / Continuous Delivery (Déploiement/ Livraison Continue)

CPU : Central Processing Unit

CSS : Cascading Style Sheets

DB : Database (Base de Données)

DNS : Domain Name System

DOCKERFILE : Fichier de configuration pour la création d'images Docker)

FUNTOA : Funhece Toamasina

GHCR : GitHub Container Registry

HTTP : HyperText Transfer Protocol

HTTPS : HyperText Transfer Protocol Secure

IP : Internet Protocol

JSON : JavaScript Object Notation

KPI : Key Performance Indicator

LAN : Local Area Network

MVC : Model–View–Controller

NAT : Network Address Translation

NGINX : Serveur web et reverse proxy

OS : Operating System (Système d'exploitation)

RAM : Random Access Memory

REST : Representational State Transfer

SQL : Structured Query Language

SMIE : Service Médical Interentreprises

SSH : Secure Shell

URL : Uniform Resource Locator

VPS : Virtual Private Server

INTRODUCTION GENERALE

L'évolution rapide des technologies de l'information et de la communication impose aujourd'hui aux organisations une adaptation continue de leurs systèmes numériques. Dans un environnement de plus en plus compétitif et axé sur la qualité de service, la modernisation des infrastructures informatiques constitue un levier essentiel pour améliorer l'efficacité, la fiabilité et la performance opérationnelle. Le Service Médical Interentreprises (SMIE), engagé dans une démarche d'optimisation de ses processus internes, s'inscrit pleinement dans cette dynamique de transformation numérique.

Le présent mémoire s'intéresse à la **modernisation technologique du système applicatif du SMIE**, à travers la conception et la mise en œuvre d'une chaîne **CI/CD** reposant principalement sur **GitHub Actions**, **Docker**, ainsi qu'un environnement de déploiement basé sur un **serveur VPS**. Cette approche permet d'automatiser l'ensemble des processus de développement, de test et de mise en production, tout en garantissant une meilleure isolation des services, une standardisation des environnements et une réduction significative des erreurs liées aux interventions manuelles.

L'objectif de ce travail est double : d'une part, proposer une architecture moderne, flexible et adaptée aux besoins opérationnels du SMIE ; d'autre part, démontrer l'apport des pratiques DevOps dans l'amélioration de la qualité logicielle et dans l'optimisation du cycle de vie applicatif. Ce mémoire présente ainsi l'étude du contexte, les choix technologiques justifiés, la conception du système, la réalisation des pipelines CI/CD, les étapes de déploiement, ainsi que l'évaluation des résultats obtenus.

À travers cette démarche, ce projet ambitionne de montrer que l'intégration cohérente d'outils DevOps et de technologies de containerisation constitue un fondement essentiel pour la modernisation des systèmes d'information, et représente une orientation stratégique pour les futures évolutions technologiques au sein du SMIE.

PARTIE I : PRESENTATIONS

Chapitre 1: Présentation de l'Ecole Nationale d'Informatique

L'École Nationale d'Informatique (ENI) est reconnue comme un établissement de référence à Madagascar dans le domaine de la formation en informatique.

1.1. Information d'ordre générale

L'Ecole Nationale d'Informatique, en abrégé ENI, est un établissement d'enseignement supérieur rattaché académiquement et administrativement à l'Université de Fianarantsoa. Le siège de l'Ecole se trouve à Tanambao-Antaninarenina à Fianarantsoa. L'adresse pour la prise de contact avec l'Ecole est la suivante : Ecole Nationale d'Informatique (ENI) Tanambao, Fianarantsoa. Le numéro de sa boîte postale est 1487 avec le code postal 301. Téléphone : 038 96 566 96 ou 034 76 591 95. Son adresse électronique est la suivante : **eni@eni.mg**. Il dispose également d'un site web : **www.eni.mg**

1.2. Missions et historiques

L'ENI se positionne sur l'échiquier socio-éducatif malgache comme étant le plus puissant secteur de diffusion et de vulgarisation des connaissances et des technologies informatiques.

Cette Ecole Supérieure peut être considérée aujourd'hui comme la vitrine et la pépinière des élites informaticiennes du pays.

De façon formelle, l'ENI était créée par le décret N° 83- 185 du 24 Mai 1983, comme étant le seul établissement Universitaire Professionnalisé au niveau national, destiné à former des techniciens et des Ingénieurs de haut niveau, aptes à répondre aux besoins et exigences d'Informatisation des entreprises, des sociétés et des organes implantés à Madagascar.

L'ENI a pour conséquent pour mission de former des spécialistes informaticiens compétents et opérationnels de différents niveaux notamment :

- En fournissant à des étudiants des connaissances de base en informatique ;
- En leur transmettant le savoir-faire requis, à travers la professionnalisation des formations dispensées et en essayant une meilleure adéquation des formations par rapport aux besoins évolutifs des sociétés et des entreprises ;
- En initiant les étudiants aux activités de recherche dans les différents domaines des Technologies de l'Information et de la Communication (TIC) ;

La filière de formation d'Analystes Programmeurs a été mise en place à l'Ecole en 1983, et a été gelée par la suite en 1996, tandis que la filière de formation d'ingénieurs a été ouverte à l'Ecole en 1986.

Une formation de troisième cycle a été ouverte à l'Ecole depuis l'année 2003 – 2004 grâce à la coopération académique et scientifique entre l'Université de Fianarantsoa pour le compte de l'ENI et l'Université Paul Sabatier de Toulouse (UPST).

Cette filière avait pour objectif de former certains étudiants à la recherche dans les différents domaines de l'Informatique, et notamment pour préparer la relève des Enseignants-Chercheurs qui étaient en poste.

Pendant l'année 2007-2008, la formation en vue de l'obtention du diplôme de Licence Professionnelle en Informatique a été mise en place à l'ENI avec les deux parcours de formation :

- Génie Logiciel et base de Données.
- Administration des Système et réseaux.

La mise en place à l'Ecole de ces deux options de formation devait répondre au besoin de basculement vers le système Licence – Master – Doctorat (LMD).

En vue de surmonter les difficultés de limitation de l'effectif des étudiants accueillis à l'Ecole, notamment à cause du manque d'infrastructures, un système de « Formation Hybride » a été mise en place à partir de l'année 2010. Il s'agit en effet d'un système de formation semi présentielle et à distance avec l'utilisation de la visioconférence pour la formation à distance. Le système de formation hybride a été ainsi créé à Fianarantsoa ainsi qu'Université de Toliara. Cette formation est à l'origine du parcours Informatique Générale.

En 2023, la mention Intelligence Artificielle (IA) a été ouvert au sein de l'Ecole pour répondre les besoins des entreprises. La formation est destinée aux étudiants titulaires du diplôme de licence (Bac +3) en Mathématiques ou en Statistiques ou en Informatique, etc. La mention IA comporte deux parcours :

- Gouvernance et Ingénierie de Données (GID),
- Objets connectés et Cybersécurité (OCC).

Le principe de l'enseignement pour le parcours GID offre aux l'étudiants des compétences scientifiques et techniques spécialisées en Science de données. Pour le parcours OCC, les étudiants octroient la double spécialité premièrement en internet des objets et deuxièmement en

cybersécurité. La formation de master est axée sur l'ensemble d'applications de l'Intelligence Artificielle.

Depuis l'année universitaire 2024-2025, la mention Expertise Digitale est officiellement ouverte au grand public. Cette mention propose deux (2) parcours distincts :

- Métiers du Digital (MDi) – disponible aux niveaux Licence et Master. Ce parcours est destiné aux étudiants titulaires du diplôme de Baccalauréat, toutes séries confondues. Il est élaboré pour équiper les étudiants des compétences essentielles dans les domaines du marketing digital, de la communication digitale, et de la gestion de projets digitaux.
- Audit des Systèmes d'Information (ASI) – proposé au niveau Master uniquement. Ce parcours s'adresse aux étudiants titulaires d'une licence (Bac +3) en Mathématiques, en Statistiques ou en Informatique. Il est structuré pour approfondir les connaissances théoriques et pratiques nécessaires à l'audit efficace des systèmes d'information.

Le système d'enseignement adopté pour cette mention est un système en ligne. Ainsi, les cours et ressources pédagogiques sont accessibles en permanence via la plateforme numérique Moodle. Par ailleurs, des regroupements en présentiel ou en ligne seront organisés périodiquement, conformément au planning défini par les responsables pédagogiques.

1.3. Organigramme institutionnel

L'organigramme de l'Ecole est inspiré des dispositions du décret N° 83-185 du 24 Mai 1983. L'ENI est administrée par un Conseil d'Ecole, et dirigée par un directeur nommé par un décret adopté en Conseil des Ministres. Le Collège des enseignants regroupant tous les enseignants-chercheurs permanents de l'Ecole est chargé de résoudre les problèmes liés à l'organisation pédagogique des enseignements. Le Conseil Scientifique propose les orientations pédagogiques et scientifiques de l'établissement, en tenant compte notamment de l'évolution du marché de travail et de l'adéquation des formations dispensées par rapport aux besoins des entreprises.

La figure 1 représente l'organigramme actuel de l'Ecole Nationale d'Informatique.

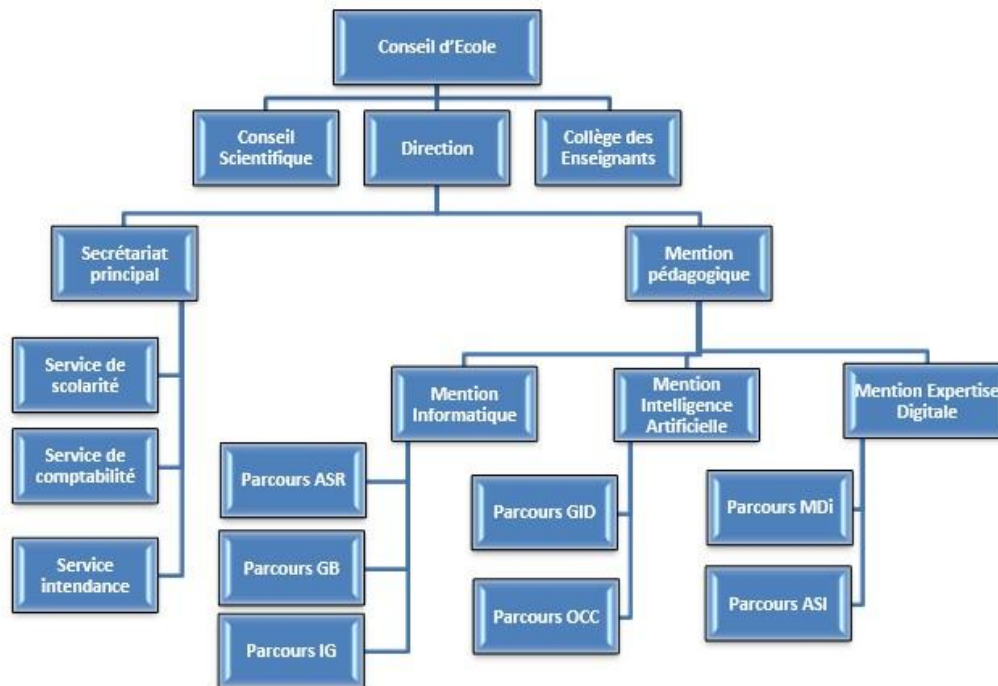


Figure 1: Organigramme de l'Ecole Nationale d'Informatique

1.4. Domaine de spécialisation

Les activités de formation et de recherche organisées à l'ENI portent sur les domaines suivants :

- Génie logiciel et Base de Données ;
- Administration des Systèmes et Réseaux ;
- Informatique Générale ;
- Modélisation informatique et mathématique des Systèmes complexes ;
- Intelligence artificielle.

Le tableau 1 décrit l'organisation du système de formation pédagogique de l'Ecole.

Tableau 1: Organisation du système de formation pédagogique de l'Ecole

Formation Théorique	Formation Pratique
➤ Enseignement théorique ➤ Travaux dirigés ➤ Travaux pratiques ➤ Conférences	➤ Etude de cas ➤ Travaux de réalisation ➤ Projets/ Projets tutorés ➤ Voyages d'Etudes ➤ Stages en entreprise

1.5. Architecture des formations pédagogiques

Le recrutement des étudiants à l'ENI se fait uniquement par voie de concours d'envergure nationale en première année. Les offres de formation organisées à l'Ecole ont été validées par la Commission Nationale d'Habilitation (CNH). Au sein de l'ENI, il existe deux mentions et cinq parcours. Le tableau 2 récapitule les mentions et les parcours au sein de l'Ecole :

Tableau 2: Mention et parcours au sein de l'ENI

Mention	Parcours
Informatique	Génie logiciel et Base de Données (GB)
	Administration des Systèmes et Réseaux (ASR)
	Informatique Générale (IG)
Intelligence Artificielle	Gouvernance et Ingénierie de Données (GID)
	Objets Connectés et Cyber sécurités (OCC)
Expertise Digitale	Métiers du Digital (MDi)
	Audit des Systèmes d'Information (ASI)

La figure 2 représente l'architecture des études correspondant au système LMD.

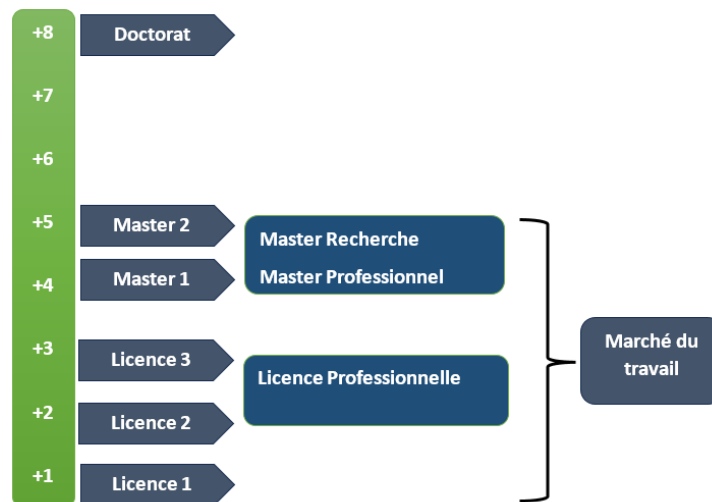


Figure 2: Architecture des études correspondant au système LMD

La licence peut avoir une vocation générale ou professionnelle. Le master peut avoir une vocation professionnelle ou de recherche. L'accès en première année de MASTER se fait automatiquement pour les étudiants de l'Ecole qui ont obtenu le diplôme de Licence Professionnelle.

Le tableau 3 illustre la liste des formations existantes à l'ENI.

Tableau 3: Liste des formations existantes à l'ENI

	FORMATION	
	LICENCE PROFESSIONNELLE	MASTER
Condition admission	Par voie de concours	Par voie de concours pour la mention IA
Condition d'accès	Bac de série C, D ou Technique	Être titulaire de licence professionnelle
Durée de Formation	3 ans	2 ans
Diplôme délivré	Diplôme de Licence Professionnelle	Diplôme de Master Professionnel Diplôme de Master Recherche

Le Master Recherche permet à son titulaire de poursuivre directement des études en doctorat et de s'inscrire directement dans une Ecole Doctorale.

Les étudiants diplômés de l'Ecole sont plutôt bien accueillis dans les instituts universitaires étrangères (Canada, Suisse, France, ...)

1.6. Relation de l'ENI avec les organismes externes

Les stages effectués chaque année par les étudiants mettent l'Ecole en rapport permanent avec plus de 400 entreprises et organismes publics, semi-publics et privés, nationaux et internationaux. L'Ecole dispose ainsi d'un réseau d'entreprises, de sociétés et d'organismes publics et privés qui sont des partenaires par l'accueil en stage de ses étudiants, et éventuellement pour le recrutement après l'obtention des diplômes par ces derniers. Les compétences que l'Ecole cherche à développer chez ses étudiants sont l'adaptabilité, le sens de la responsabilité, du travail en équipe, le goût de l'expérimentation et l'innovation.

En effet, la vocation de l'ENI est de former des licenciés et des ingénieurs de niveau MASTER avec des qualités scientifiques, techniques et humaines reconnues, capables d'évoluer professionnellement dans des secteurs d'activité variés intégrant l'informatique. Les stages en milieu professionnel permettent de favoriser une meilleure adéquation entre les formations à l'Ecole et les besoins évolutifs du marché de l'emploi.

Parmi les sociétés, les entreprises et les organismes partenaires de l'Ecole, on peut citer : ACCENTURE Mauritius, AKATA Goavana, Air Madagascar, Ambre Associates, Airtel, Agence Universitaire de la Francophonie (AUF), AXIAN, B2B, Banque Centrale, , BIANCO, BlueLine, CNaPS, Bureau National de Gestion des Risques et des Catastrophes (BNGRC), CEDII-Fianarantsoa, Data Consulting, Central Test, Centre National Antiacridien, CNRE, COLAS, Direction Générale des Douanes, DLC, E-Tech Consulting, , FID, FIHARY Soft, FTM, GNOSYS, GENIUS AT WORK, Hello Tana, IBONIA, INGENOSIA, INSTAT, IOGA, JIRAMA, JOUVE, MADADEV, MAEP, MANAO, MEF, MEN, MESupRES, MFB, , MININTER, Min des Postes/Télécommunications et du Développement Numérique, NEOV MAD, Ny Havana, Madagascar National Parks, OMNITEC, ORANGE, OTME, PRACCESS, QMM Fort-Dauphin, SG Madagasikara SMMC, SMMEC, SNEDADRS Antsirabe, Sénat, Société d'Exploitation du Port de Toamasina (SEPT), SOFTWELL, Strategy Consulting, TELMA, VIVETEC, Société LAZAN'I BETSILEO, WWF, UGD, ARATO, MANAO, MNDPT, NG ACADEMY.NG, Relia, Spoon Consulting ...

1.7. Débouchés professionnels et diplômés

Les formations proposées par l'Ecole permettent aux diplômés d'être immédiatement opérationnels sur le marché du travail avec la connaissance d'un métier complet lié à l'informatique aux TIC.

L'Ecole apporte à ses étudiants un savoir-faire et un savoir-être qui les accompagnent tout au long de leur vie professionnelle. Elle a une vocation professionnalisante. Les diplômés en LICENCE et en MASTER issus de l'ENI peuvent faire carrière dans différents secteurs.

L'Ecole bénéficie aujourd'hui de 40 années d'expériences pédagogiques et de reconnaissance auprès des sociétés, des entreprises et des organismes. C'est une Ecole Supérieure de référence en matière informatique.

D'une manière générale, les diplômés de l'ENI n'éprouvent pas de difficultés particulières à être recrutés au terme de leurs études. Cependant, l'ENI recommande à ses diplômés de promouvoir l'entrepreneuriat en TIC et de créer des cybercafés, des SSII ou des bureaux d'études.

Le tableau 4 représente les débouchés éventuels selon la mention.

Tableau 4: Débouchés éventuels des jeunes diplômés

Mention	Débouchés professionnels
INFORMATIQUE	GB : Développeur logiciel / développeur full-stack, Ingénieur en développement d'applications (desktop, web, mobile), Architecte logiciel, Intégrateur d'applications, Développeur d'API, Administrateur de bases de données (DBA), Ingénieur en conception de bases de données, Développeur SQL / PL-SQL, Spécialiste en migration et optimisation de bases de données, Data engineer (ingénieur en traitement de données), Analyste programmeur, Analyste fonctionnel, Chef de projet informatique, Scrum master / Product owner (en méthodologie Agile), Ingénieur QA (Quality Assurance), Testeur logiciel / testeur automatisé, Spécialiste maintenance applicative, Expert en refactoring et optimisation de code, Analyste de données (Data analyst), Concepteur d'outils décisionnels (BI), Ingénieur en Big Data, Machine Learning Engineer, Consultant fonctionnel ou freelance ASR : Administrateur systèmes et réseaux, Ingénieur systèmes et réseaux, Administrateur systèmes Linux / Windows, Administrateur cloud (AWS, Azure, Google Cloud), Ingénieur virtualisation (VMware, Hyper-V, Proxmox), Administrateur sécurité (Security Administrator), Ingénieur cybersécurité, Analyste SOC (Security Operations Center), Spécialiste firewall / VPN, Responsable sécurité informatique (RSSI – avec expérience), Technicien support informatique (N2/N3), Technicien réseaux et télécoms, Ingénieur exploitation, Administrateur messagerie (Exchange, Microsoft 365), Technicien datacenter, Architecte réseaux, Architecte systèmes, Consultant infrastructure, Chef de projet systèmes et réseaux, Ingénieur DevOps, Administrateur Kubernetes / Docker, Cloud engineer, Spécialiste automatisation (Ansible, Terraform) IG : GB et SR en même temps

INTELLIGENCE ARTIFICIELLE	GID : Développeur IA, Data Scientist, Entrepreneur IA, Consultant IA, Responsable marketing IA, Business Developer IA, Product Owner, Consultant en cybersécurité, Directeur de l'Innovation ; OCC : Analyste en cybersécurité, Ingénieur en cybersécurité, Spécialiste en réponse aux incidents, Architecte de sécurité, Consultant en cybersécurité, Auditeur en cybersécurité, Expert en sécurité des données, Responsable de la sécurité des informations, Chercheur en cybersécurité, Formateur en cybersécurité, Ingénieur IoT, Développeur IoT, Architecte IoT, Spécialiste en sécurité IoT, Analyste de données
EXPERTISE DIGITALE	MDi : Spécialiste en Marketing Digital, Gestionnaire de Contenu Web, Analyste UX/UI, Product Owner, Chef de Projet Digital, Stratège en Médias Sociaux, Analyste de Performance Digitale, Chef de Projet Digital, Stratège en Médias Sociaux, Concepteur-Rédacteur Web Analyste de Performance Digitale, Expert en Commerce Électronique, Expert en Commerce Électronique, Spécialiste SEO/SEM, Manager de Communauté (Community Manager), Conseiller en Stratégie Digitale, Responsable CRM (Customer Relationship Management), Spécialiste en Analytique et Intelligence d'Affaires (Business Intelligence), Consultant en Transformation Digitale, Responsable des Partenariats Digitaux, Spécialiste en Cybersécurité Marketing, Directeur de l'Innovation Digitale, Manager de l'E-réputation ASI : Auditeur des Systèmes d'Information, Consultant en Sécurité des Systèmes d'Information, Analyste de Risques IT, Chef de Projet SI, Responsable de la Conformité IT, Spécialiste en Cyber Forensics, Directeur de l'Audit et de la Sécurité des Systèmes d'Information, Expert en Gouvernance IT, Formateur ou Éducateur en Sécurité des Chercheur en Sécurité IT

1.8. Ressources humaines

Les ressources humaines sont citées ci-dessous selon leurs responsabilités :

- Directeur de l'Ecole : Monsieur MAHATODY Thomas, Professeur
- Responsable de la Mention « Informatique » : Monsieur RALAIVAO Jean Christian, Maître de Conférences
- Responsable de la Mention « Intelligence Artificielle » : Monsieur DIMBISOA William Germain, Maître de Conférences

L'ENI compte douze (12) enseignants permanents dont un (01) Professeur Titulaire, deux (02) Professeurs, Sept (07) Maîtres de Conférences, Deux (02) Assistants d'Enseignement Supérieur et de Recherche, dix (10) enseignants vacataires, quarante un (41) personnel administratif.

Chapitre 2: Présentation de SMIE FUNTOA

Le **Service Médical Interentreprise FUNTOA (SMIE FUNTOA)** est un organisme de santé agréé, spécialisé dans la prise en charge médicale des travailleurs salariés des entreprises adhérentes. Implanté stratégiquement à Toamasina, ville portuaire et centre économique majeur de Madagascar, FUNTOA SMIE s'impose comme un acteur incontournable dans la médecine du travail et la prévention sanitaire en entreprise.

La structure se distingue par la qualité de ses prestations, la diversité de ses services et son engagement constant en faveur du bien-être des employés. Grâce à des équipements modernes, une équipe médicale qualifiée et un réseau de partenaires à l'échelle nationale, FUNTOA SMIE fournit une couverture médicale complète, conforme aux exigences réglementaires tout en intégrant progressivement des solutions innovantes de digitalisation.

2.1. Statut légal et identification de l'organisation

SMIE FUNTOA bénéficie d'une existence légale pleinement reconnue par les autorités compétentes.

Il dispose :

- **D'un agrément officiel N° 221-14/MFPTLS/SG/DGTLS du 18/12/2014**, délivré par le Ministère chargé du Travail, autorisant l'exercice de ses activités en tant que service médical interentreprise ;
- **D'un Numéro d'Identification Fiscale (NIF) : 400 388 5426**, permettant son immatriculation et sa conformité aux obligations fiscales ;
- **D'un code statistique : 86909 31 2019 0 01640**, attestant de son enregistrement au niveau national.

Ces éléments administratifs confirment que SMIE FUNTOA agit dans un cadre réglementaire strict, conforme aux normes en vigueur sur la médecine du travail, la protection sanitaire des salariés et la gestion des données médicales.

L'organisation est implantée à l'adresse suivante :

📍 **Lot 1599 Carreau II, Parcelle 11/14 – Mangarivotra Nord, Toamasina (501), Madagascar**

Ce positionnement géographique est particulièrement stratégique, Toamasina étant l'un des plus importants pôles industriels et logistiques du pays. La proximité de FUNTOA avec les entreprises locales facilite l'accès aux services médicaux pour les travailleurs, optimise les interactions avec les partenaires et renforce l'efficacité des prestations offertes.

SMIE FUNTOA assure également une accessibilité accrue à travers différents moyens de communication :

- Téléphones : **032 07 366 11, 032 07 371 40, 020 76 008 48**
- Adresse électronique : funtoasmie@gmail.com
- Site web : www.funtoa-smie.com

2.2. Vision, mission et philosophie d'intervention

La vision de SMIE FUNTOA repose sur l'idée que **la santé des collaborateurs représente l'un des piliers fondamentaux de la performance des entreprises.**

Sa devise :

« Santé et Sécurité : main dans la main vers le succès » reflète son ambition de créer une synergie positive entre bien-être au travail, productivité et développement économique.

2.2.1. Mission principale

La mission centrale de FUNTOA est d'assurer une **surveillance médicale préventive et curative**, conforme aux exigences du Code du Travail.

Elle englobe :

- La prévention des risques professionnels,
- La promotion de la santé en entreprise,
- L'amélioration de la qualité de vie au travail,
- La réduction de l'absentéisme,
- Le suivi régulier des salariés.

2.2.2. Philosophie et valeurs

FUNTOA fonde son intervention sur plusieurs valeurs fondamentales :

- **Éthique** : confidentialité totale des données médicales et respect de la déontologie.
- **Responsabilité** : engagement pour un environnement de travail sain et sécurisé.
- **Innovation** : intégration progressive d'outils numériques modernes (tableaux de bord, reporting), préparant le terrain à la transformation digitale du système d'information.
- **Accessibilité** : garantir un accès rapide, fiable et continu aux soins.

2.3. Entreprises partenaires et réseau d'intervention nationale

Ces valeurs positionnent l'institution comme un acteur sérieux, crédible et responsable.

SMIE FUNTOA collabore avec une large gamme d'entreprises opérant dans des secteurs variés : industrie, logistique, finance, télécommunication, restauration, etc. Parmi les entreprises partenaires les plus influentes, on peut citer :

- **CONCENTRIX Tamatave**
- **GALANA Raffinerie Terminal (GRT)**
- **BFV-SG**
- **ACCES Banque**
- **OTI**
- **Gastronomie Pizza**

Cette diversité de partenariats démontre la solidité du service et la confiance que les entreprises accordent à FUNTOA.

En complément, SMIE FUNTOA bénéficie d'un **réseau national structuré** incluant :

- FUNHECE Antananarivo,
- FUNFIA Fianarantsoa,
- FUNABE Antsirabe,

- Ainsi que plusieurs médecins prestataires répartis dans tout Madagascar.

2.4. Ressources humaines et organisation interne

Ce réseau offre une couverture nationale permettant aux salariés d'accéder facilement aux soins, même lors de déplacements professionnels ou de mutations.

L'efficacité de SMIE FUNTOA repose sur une équipe pluridisciplinaire hautement qualifiée, comprenant :

- Des médecins généralistes,
- Des spécialistes,
- Des infirmiers expérimentés,
- Des pharmaciens,
- Des techniciens de laboratoire,
- Du personnel administratif formé aux procédures de gestion sanitaire.

Cette équipe est opérationnelle **24h/24 et 7j/7**, garantissant une continuité de service exceptionnelle. Le caractère permanent des soins est essentiel pour les entreprises fonctionnant en horaires décalés ou en production continue (industrie, port, transport, etc.).

L'organisation interne est structurée de manière à assurer :

- Une coordination optimale entre les différents services médicaux,
- Une prise en charge rapide des urgences,
- Une fluidité des circuits de consultation,
- Une réactivité face aux demandes des entreprises partenaires.

2.5. Atouts stratégiques et avantages concurrentiels

SMIE FUNTOA se distingue par un ensemble d'atouts qui renforcent sa position dans le domaine de la médecine du travail :

2.5.1. Compétitivité financière

Le tarif d'adhésion est fixé à :

- **1 % de la masse salariale** (frais d'entrée),
- Avec des cotisations mensuelles de **6 % du salaire brut** (5 % employeur + 1 % salarié).

Ce tarif reste l'un des plus compétitifs de la région tout en garantissant des prestations de haute qualité.

2.5.2. Infrastructures médicales modernes et diversifiées

Le centre dispose de :

- Pharmacie 24/7,
- Laboratoire complet,
- Service d'imagerie,
- Salle d'urgence,
- Service de perfusion,
- Salles de consultation modernes et équipées.

2.5.3. Réactivité et excellence opérationnelle

- traitement rapide des doléances (moins de 24h),
- communication transparente avec les entreprises,
- tableaux de bord interactifs,
- suivi statistique avancé.

2.5.4. Engagement en matière de qualité et d'éthique médicale

Les médicaments distribués proviennent exclusivement de fournisseurs agréés par l'Agence du Médicament de Madagascar, assurant qualité, sécurité et conformité.

2.6. Offre globale des services médicaux

2.6.1. Couverture médicale complète

Incluant :

- Consultations généralistes et spécialisées,
- Soins paramédicaux (pansements, injections, perfusions, petite chirurgie),
- Dentisterie d'urgence,

- Pharmacie accessible 24/7.

2.6.2. Services exclusifs

- urgences médicales permanentes,
- salle de surveillance et perfusion,
- services maternels (suivi prénatal, pesée des nouveau-nés),
- programme complet de vaccination.

2.6.3. Médecine du travail et prévention

- visites d'embauche, systématiques, de reprise,
- bilans annuels de santé,
- campagnes de prévention (VIH/SIDA, stress, secourisme),
- reporting mensuel détaillé.

2.6.4. Gestion des doléances

Trois canaux dédiés :

- Registre officiel,
- E-mail,
- Boîte à idées.

2.7. Perspective de transformation digitale

Face à l'évolution constante du volume d'entreprises partenaires et à la complexité croissante des données à gérer, FUNTOA SMIE exprime un besoin concret de **modernisation de son système d'information**.

Les attentes principales concernent :

- La centralisation des données,
- La numérisation des dossiers médicaux,

- La sécurisation des informations sensibles,
- La réduction des traitements manuels,
- La fiabilité du suivi médical,
- La génération automatique de rapports.

Le présent projet de mémoire, basé sur le développement d'un système d'information moderne accompagné d'un pipeline CI/CD, vise à répondre directement à ces nouveaux enjeux.

SMIE FUNTOA est aujourd'hui l'un des acteurs les plus importants dans le domaine de la médecine du travail à Toamasina. Par son réseau, ses infrastructures, ses ressources humaines et sa mission, il occupe une place stratégique dans la santé des travailleurs. Cependant, pour atteindre son plein potentiel et accompagner efficacement sa croissance, l'organisation doit s'appuyer sur un système d'information moderne, automatisé et conforme aux standards technologiques actuels. La transformation numérique représentée dans ce mémoire constitue donc une étape essentielle pour soutenir les ambitions de FUNTOA et améliorer durablement la qualité de ses services.

Chapitre 3: Présentation et cadrage du projet

3.1. Contexte général du projet

Le Service Médical Interentreprises s'appuyait principalement sur des outils bureautiques, notamment des fichiers Excel, pour gérer ses activités administratives et médicales. Cette organisation, bien que fonctionnelle au départ, présentait des limites importantes : absence de centralisation, risques d'erreurs, faible traçabilité, partage complexe des informations et impossibilité d'automatiser les processus. Face à ces contraintes, l'organisme a exprimé la volonté de moderniser son fonctionnement et de disposer d'un système d'information fiable, cohérent et évolutif, capable de soutenir ses besoins opérationnels croissants.

C'est dans ce contexte que se situe mon stage de fin d'études, orienté vers la **modernisation du système de gestion des services médicaux** et l'adoption d'une démarche DevOps. Mon rôle a consisté à analyser l'existant, concevoir et développer une solution modernisée basée sur **Node.js**, **React** et **MySQL**, puis à industrialiser son déploiement à l'aide de **Docker**, **GitHub Actions** et d'un **serveur VPS**. L'objectif global était de remplacer les fichiers Excel par une application centralisée et automatisée, tout en assurant une intégration continue, une meilleure fiabilité des données et une gestion opérationnelle plus efficace.

3.2. Objectif et besoins de l'utilisateur

Le projet de modernisation du système de gestion des services médicaux vise à répondre aux limitations rencontrées dans l'état initial, où les opérations étaient réalisées uniquement à l'aide d'outils bureautiques tels qu'Excel. Cette approche, bien que fonctionnelle pour un petit volume de données, ne permettait pas de gérer efficacement les services médicaux interentreprises, entraînant des difficultés de centralisation, de suivi et de sécurisation des informations.

L'objectif principal du projet est donc de fournir aux utilisateurs un **système d'information intégré et automatisé**, capable de centraliser toutes les données relatives aux services médicaux et de simplifier la gestion des opérations quotidiennes. Concrètement, le projet vise à :

- **Optimiser la gestion des données médicales** : les utilisateurs doivent disposer d'une base de données centralisée, fiable et facilement accessible, leur permettant de consulter, modifier et analyser les informations de manière sécurisée et structurée.

- **Automatiser les processus administratifs** : l'application doit réduire le recours aux saisies manuelles et aux tâches répétitives, en automatisant les workflows et en générant automatiquement certains rapports et alertes.
- **Assurer un suivi en temps réel** : le système doit permettre aux responsables et aux professionnels de santé de suivre instantanément l'état des dossiers, la disponibilité des services et les indicateurs clés de performance.
- **Offrir une interface conviviale et intuitive** : l'ergonomie de l'application est primordiale afin que chaque utilisateur, quel que soit son niveau technique, puisse interagir avec le système facilement et efficacement.
- **Garantir la sécurité et la confidentialité des données** : le système doit intégrer des mécanismes de sécurisation avancés pour protéger les informations médicales sensibles, conformément aux normes en vigueur et aux exigences réglementaires.

Ainsi, le projet ne se limite pas à une simple modernisation technique, mais répond directement aux besoins fonctionnels et opérationnels des utilisateurs, en offrant un outil complet, fiable et sécurisé pour la gestion des services médicaux.

3.3. Moyens nécessaires à la réalisation du projet

La réalisation du projet de modernisation du système de gestion des services médicaux nécessite l'utilisation de moyens techniques, humains et organisationnels adaptés. Ces moyens permettent d'assurer le développement, le déploiement et le fonctionnement optimal de l'application, tout en répondant aux exigences des utilisateurs.

3.3.1. Moyens techniques

Pour atteindre les objectifs du projet, plusieurs technologies et outils ont été mobilisés. Leur choix a été guidé par la nécessité d'assurer une application performante, sécurisée, facile à maintenir et évolutive.

Environnement de développement :

Node.js a été utilisé pour le développement du backend. Cette technologie permet de créer des applications serveur performantes grâce à son moteur JavaScript asynchrone et événementiel. Node.js offre également une grande compatibilité avec les bases de données modernes et facilite l'intégration d'API.

React a été choisi pour le frontend afin de créer une interface utilisateur réactive, modulable et performante. Sa capacité à réutiliser des composants permet un développement rapide et une maintenance simplifiée.

MySQL a été employé pour la gestion de la base de données. Ce système de gestion relationnelle garantit la fiabilité, l'intégrité et la sécurité des données. Il permet également de structurer efficacement les informations médicales et de réaliser des requêtes complexes pour les rapports et analyses.

Conteneurisation et déploiement :

Docker a été utilisé pour containeriser l'application. Chaque composant du système (backend, frontend, base de données) est isolé dans un conteneur, assurant ainsi la cohérence entre les environnements de développement, de test et de production. Cette approche facilite également le déploiement, la scalabilité et la portabilité de l'application sur différents serveurs.

Intégration et livraison continues (CI/CD) :

GitHub Actions a été mis en place pour automatiser les processus de build, de test et de déploiement. Cette automatisation réduit les risques d'erreurs humaines, permet un suivi constant de la qualité du code et accélère le cycle de livraison, garantissant ainsi des mises à jour fiables et rapides de l'application.

Serveur VPS (Virtual Private Server) :

L'application est hébergée sur un VPS, assurant une disponibilité continue et un accès sécurisé pour tous les utilisateurs. Ce type d'hébergement offre un contrôle complet sur l'environnement serveur, la configuration réseau et les mesures de sécurité, ce qui est essentiel pour la gestion de données médicales sensibles.

Chaque technologie et outil choisi contribue ainsi à répondre aux besoins fonctionnels et techniques du projet, tout en garantissant la robustesse, la sécurité et l'évolutivité de l'application.

3.3.2. Moyens humains

La réussite du projet de modernisation du système de gestion des services médicaux repose principalement sur les compétences et l'implication directe du stagiaire, en l'occurrence le

développeur principal du projet, ainsi que sur les échanges réguliers avec le directeur de la société, représentant les besoins utilisateurs.

➤ **Le stagiaire :**

Le rôle central dans le projet a été assumé par le stagiaire, chargé de **moderniser technologiquement le système existant** en développant une application complète pour la gestion des services médicaux. Ses missions ont couvert l'ensemble du cycle de vie de l'application, incluant le **développement backend et frontend**, la conception et la structuration de la base de données, ainsi que la mise en place d'une **pipeline CI/CD** pour automatiser le déploiement et assurer une livraison fiable sur le serveur VPS. De plus, le stagiaire a pris en charge l'administration du serveur, la gestion du réseau et le maintien de l'environnement de production, démontrant ainsi une autonomie complète dans la gestion technique du projet.

➤ **Le directeur de la société :**

Bien que n'intervenant pas directement dans le développement technique, le directeur a joué un rôle clé en **fournissant les besoins fonctionnels et en validant la conception de l'application**. Ses orientations ont permis de s'assurer que le système développé répondait aux exigences réelles des services médicaux et était conforme aux attentes opérationnelles de l'entreprise.

➤ **Autonomie et responsabilité :**

L'ensemble du projet a été réalisé de manière autonome par le stagiaire, incluant à la fois **la conception, le développement, le déploiement et la maintenance technique**. Cette configuration a nécessité une maîtrise complète des technologies utilisées et une capacité à prendre des décisions techniques adaptées aux contraintes du projet.

Ainsi, les moyens humains du projet sont concentrés autour d'une **forte autonomie du stagiaire**, soutenue par des échanges fonctionnels avec le directeur de la société, garantissant la réalisation d'un système d'information complet et répondant aux besoins des utilisateurs.

3.3.3. Moyens matériels

Avant de procéder à la réalisation technique du projet, il a été nécessaire de disposer de moyens matériels fiables et performants. Ces équipements permettent de garantir un environnement stable pour le développement, le déploiement, la gestion des conteneurs Docker, ainsi que pour l'administration du serveur et la sécurisation des données. Le choix de ces matériels a été guidé par la nécessité d'assurer la performance, la disponibilité et la sécurité de l'application, tout en facilitant le travail quotidien du développeur.

Les principales caractéristiques des moyens matériels mobilisés pour ce projet sont synthétisées dans le tableau ci-dessous.

Tableau 5 : Configuration des moyens matériels pour la réalisation du projet

Catégorie	Équipement / Configuration	Rôle dans le projet
Poste de travail	Processeur : Intel Core i5 7 ^e génération RAM : 8 Go DDR4 Stockage : SSD 126 Go + HDD 500 Go Système : Dual boot Windows 11 / Kali Linux	Développement backend (Node.js), frontend (React), gestion base de données (MySQL), tests locaux, administration du serveur et conteneurs Docker
Serveur VPS	RAM : 12 Go CPU : 6 vCPU Stockage : NVMe 100 Go	Hébergement de l'application, déploiement via Docker, exécution de la pipeline CI/CD, accès aux utilisateurs
Réseau	Connexion Wi-Fi Fibre optique Telma	Transfert des données, déploiement distant, accès sécurisé aux services et communication avec les utilisateurs

3.3.4. Moyens logiciels

La réalisation du projet de modernisation du système de gestion des services médicaux a nécessité l'utilisation de plusieurs logiciels et outils spécialisés, permettant de couvrir l'ensemble des étapes du développement, du déploiement et de la gestion du système. Ces moyens logiciels se répartissent en plusieurs catégories, selon leur rôle dans le projet.

Outils de développement :

- **VS Code** a été utilisé comme éditeur de code principal, offrant une interface flexible et de nombreuses extensions adaptées au développement en **Node.js** pour le backend et **React.js** pour le frontend.
- **Git** et **GitHub** ont été utilisés pour la gestion de version et le suivi du code source, permettant une collaboration efficace et une traçabilité complète des modifications.

Bases de données :

- **MySQL** a été choisi pour la gestion des données médicales. L'administration a été effectuée via **phpMyAdmin** pour la gestion graphique et via le terminal/CLI pour les opérations sur le serveur VPS. Cela a permis de maintenir une base de données structurée, sécurisée et performante.

Outils de conteneurisation et déploiement :

- **Docker** et **Docker Compose** ont été utilisés pour la containerisation de l'application, garantissant la portabilité et la cohérence entre les environnements.
- **GitHub Actions** a été configuré pour automatiser le pipeline CI/CD, incluant les tests, le build et le déploiement sur le serveur VPS.

Outils de tests et validation :

- **Jest** a été utilisé pour les tests unitaires et fonctionnels, permettant de vérifier le bon fonctionnement du code backend et frontend.
- **Codecov** a été intégré pour le suivi de la couverture de tests, assurant une qualité de code optimale et facilitant la maintenance future de l'application.

Logiciels de conception et modélisation :

- **Visual Paradigm** a été utilisé pour la conception du système et la modélisation UML, permettant de représenter graphiquement l'architecture de l'application, les flux de données et les interactions entre les différents composants.

Logiciels complémentaires :

- **Bitwise SSH Client** a été utilisé pour accéder au serveur VPS et gérer les configurations à distance.
- **Docker Desktop** a été utilisé pour faciliter le développement et le test local des conteneurs Docker.
- **Microsoft Word, Excel et PowerPoint** ont été utilisés pour la rédaction du mémoire, la gestion des exigences, la documentation technique et la préparation de présentations professionnelles.

L'ensemble de ces moyens logiciels a permis de créer un environnement complet, performant et sécurisé pour le développement, le déploiement et la maintenance du système, garantissant la qualité du produit final et la satisfaction des besoins des utilisateurs.

3.4. Résultats attendus

Le projet de modernisation du système de gestion des services médicaux vise à produire des résultats significatifs à la fois sur les plans fonctionnel, technique et pour les utilisateurs finaux. L'objectif principal est de transformer un système basé sur des outils bureautiques en une application complète, moderne et automatisée, tout en mettant en œuvre les pratiques DevOps à travers une pipeline CI/CD.

Sur le plan fonctionnel, l'application doit permettre une gestion intégrale des dossiers médicaux et du suivi des services, tout en offrant la possibilité de gérer le stock de la pharmacie et de générer automatiquement des rapports synthétiques pour les responsables. L'interface utilisateur se veut intuitive et moderne, de manière à faciliter l'accès aux différentes fonctionnalités sans nécessiter de compétences techniques avancées. Ces améliorations permettront aux utilisateurs de centraliser les informations, de gagner du temps dans la gestion quotidienne et d'assurer un accès rapide et sécurisé aux données depuis différents postes ou à distance.

Sur le plan technique, le projet met particulièrement l'accent sur la mise en place d'une pipeline CI/CD complète et opérationnelle. Cette pipeline permet de valider automatiquement le code développé, d'exécuter les tests unitaires et fonctionnels pour garantir la qualité du logiciel, puis de pousser les modifications sur le dépôt et enfin de déployer l'application sur le serveur VPS. L'utilisation des conteneurs Docker assure la cohérence et la portabilité de l'application entre les environnements de développement, de test et de production, tandis que le serveur VPS garantit une disponibilité continue et un accès sécurisé aux services. Cette approche permet non seulement un développement itératif et maîtrisé des fonctionnalités, mais aussi une livraison continue, réduisant les risques d'erreurs et facilitant la maintenance et l'évolution future du système.

En résumé, les résultats attendus du projet vont au-delà de la simple mise en place d'une application ; ils incarnent une **modernisation technologique complète**, intégrant des pratiques **DevOps** et un processus de développement automatisé, tout en répondant aux besoins fonctionnels et opérationnels des utilisateurs. L'application finale doit ainsi représenter un système fiable, performant et évolutif, capable de soutenir efficacement la gestion des services médicaux interentreprises.

3.5. Chronogramme des activités

Le chronogramme ci-dessous présente de manière synthétique les principales activités réalisées durant le stage, sur une durée de cinq mois, en combinant formation et mise en pratique dans le cadre du projet de modernisation du système de gestion des services médicaux.

Tableau 6 : Chronogramme des activités du stage

Période	Activités réalisées	Description détaillée
03 juillet 2024	Arrivée et intégration	Prise de contact avec l'équipe, découverte de la structure et du fonctionnement interne.
04 juillet 2024	Réunion avec le Directeur Général	Définition des objectifs du stage, compréhension des attentes et cadrage du périmètre du projet de modernisation du système d'information.
07 – 18 juillet 2024	Phase de planification	Analyse préliminaire, compréhension des besoins fonctionnels, étude du système existant (Excel, outils bureautiques) et élaboration du plan d'action.

21 – 24 juillet 2024	Finalisation du plan de travail	Structuration du workflow, définition des livrables, choix préliminaires des solutions techniques à étudier.
28 juillet – 04 août 2024	Phase de recherche technique	Recherche sur les architectures modernes (DevOps, CI/CD, conteneurisation), choix de frameworks potentiels, analyse des contraintes du contexte FUNTOA SMIE.
04 – 11 août 2024	Intervention humaine & préparation événement	Participation logistique à la préparation de l'inauguration de la nouvelle annexe de FUNTOA SMIE.
11 août 2024	Inauguration officielle	Présence et contributions à l'événement institutionnel.
13 – 22 août 2024	Choix des outils du Système d'Information	Sélection des technologies : Node.js, React.js, MySQL, Nginx, Docker, GitHub, GitHub Actions, GHCR. Définition de l'architecture cible.
25 août – 05 septembre 2024	Reconfiguration réseau & prise de contrôle	Refonte du réseau interne de la société, configuration des équipements Ubiquiti, sécurisation des accès, optimisation de la topologie réseau.
08 – 12 septembre 2024	Formation DevOps (Coursera) *essai gratuit	Suivi d'une formation intensive (une semaine) axée sur les pratiques DevOps, CI/CD, conteneurisation, automatisation.
15 – 19 septembre 2024	Pratique des concepts DevOps	Mise en application : pipelines simples, déploiements automatisés en environnement local et tests des outils étudiés.
22 – 26 septembre 2024	Recherche avancée & choix des outils CI/CD	Analyse comparée des solutions CI/CD (GitHub Actions, GitLab CI). Retenue de GitHub Actions et GHCR pour la chaîne CI/CD.
29 septembre – 21 novembre 2024	Mise en place du pipeline CI/CD & amélioration continue	Développement complet de la chaîne DevOps : Dockerisation Backend/Frontend, création du docker-compose.prod.yml, configuration GHCR,

		création des workflows (build, push, deploy), tests et optimisation.
03 – 28 novembre 2024	Rédaction du mémoire	Rédaction structurée du rapport

Afin de mieux visualiser l'organisation du travail effectué durant le stage, le diagramme de Gantt suivant retrace l'ensemble des phases principales du projet et leurs périodes de réalisation

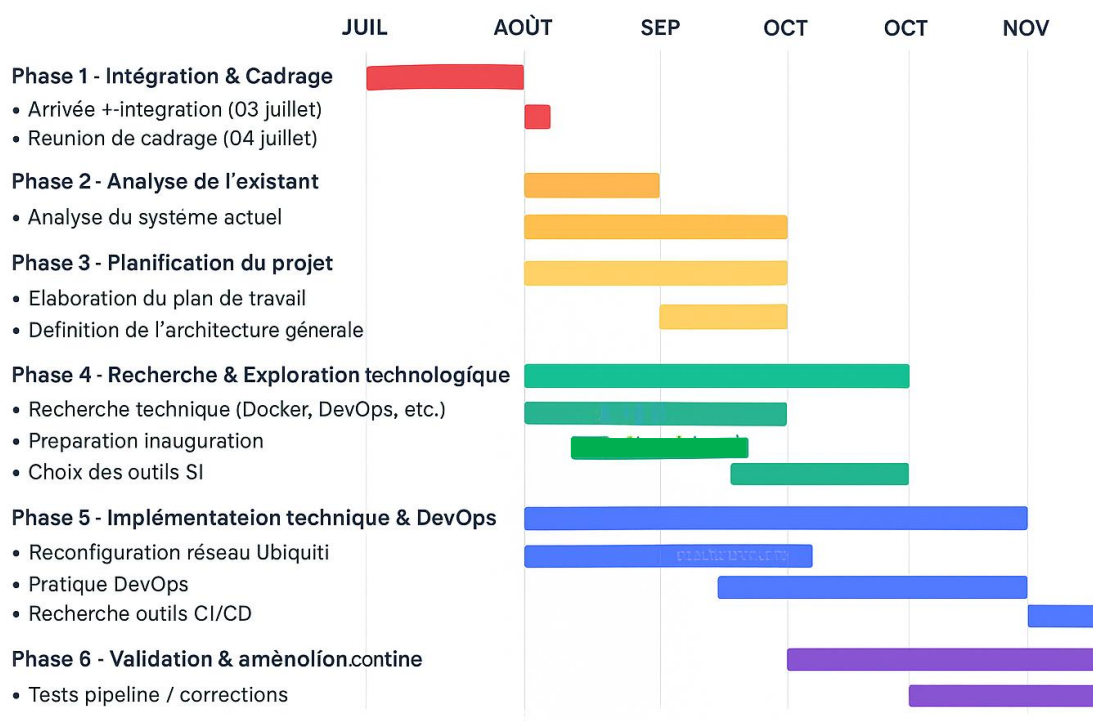


Figure 3 : Diagramme de GANTT

PARTIE II : ANALYSE ET CHOIX DES OUTILS

Chapitre 4: Analyse préalable et conception fonctionnelle

Avant de commencer le développement de l'application et la mise en place d'une pipeline CI/CD, il est indispensable d'analyser le système existant. Cette étape permet non seulement de **comprendre le fonctionnement actuel et d'identifier ses limites**, mais aussi de préparer la modernisation technologique du système en intégrant des pratiques DevOps. L'analyse de l'existant constitue ainsi le point de départ pour définir les améliorations nécessaires, planifier le déploiement automatisé et garantir un développement structuré et efficace des nouvelles fonctionnalités.

4.1. Analyse de l'existant

Avant de procéder au développement et à la modernisation du système, il est essentiel d'analyser le fonctionnement actuel afin d'identifier ses limites et d'envisager les améliorations nécessaires. Le système existant repose principalement sur **l'utilisation d'outils bureautiques**, notamment Microsoft Excel, pour la gestion des dossiers médicaux, le suivi des services et la gestion du stock de la pharmacie. Cette analyse se décline selon deux axes principaux : l'organisation actuelle et l'inventaire des moyens matériels et logiciels.

4.1.1. Organisation actuelle

Le système actuel est caractérisé par une **gestion locale et non centralisée des informations**. Chaque service ou utilisateur conserve ses propres fichiers Excel sur des ordinateurs individuels, sans mécanisme de synchronisation ni d'accès partagé. Cette organisation entraîne des **doublons, des incohérences et une difficulté de suivi**, rendant la consolidation des informations laborieuse et sujette aux erreurs.

De plus, le système ne permet pas de **suivi en temps réel** des activités ni de génération automatique de rapports. Les responsables doivent passer par des traitements manuels pour analyser les données, ce qui entraîne des **retards dans la prise de décision** et augmente le risque d'erreurs humaines. L'ensemble de ces contraintes révèle une **organisation inadéquate** pour un fonctionnement efficace et sécurisé des services médicaux.

4.1.2. Inventaire des moyens matériels et logiciels

L'inventaire des moyens matériels et logiciels permet de dresser un état clair des ressources disponibles et de préparer la modernisation du système.

a) Postes de travail et périphériques

Le parc informatique se compose d'ordinateurs de bureau et portables avec les configurations suivantes :

Tableau 7 : Configuration des ordinateurs existants

Type d'ordinateur	Processeur	RAM	Stockage	Système d'exploitation
All-in-One (04)	Core i5 4 ^e génération	4 Go	500 Go HDD	Windows 10
All-in-One (05)	Core i3 7 ^e génération	8 Go	500 Go HDD	Windows 10
Portable (01)	Core i3 7 ^e génération	4 Go	500 Go HDD	Windows 10
Portable (01)	Core i5 7 ^e génération	8 Go	500 Go HDD + 128 Go SSD	Windows 10

Le parc d'impression comprend trois imprimantes :

Tableau 8 : Inventaire des imprimantes

Marque / Modèle	Type	Remarques
Canon Série G3000	Distant	Accessible à distance
Epson	Local	Bureau principal

b) Logiciels utilisés

Les activités actuelles reposent sur les logiciels bureautiques standards : **Excel, Word, Publisher et PowerPoint**, utilisés pour la gestion des dossiers, le suivi des services et la production de rapports. Ces outils ne permettent ni la centralisation ni l'automatisation des données.

c) Réseau et connectivité

Le réseau repose sur la **fibre Telma** , avec des points d'accès **Ubiquiti** pour le Wi-Fi. Les ordinateurs et imprimantes sont connectés via LAN et Wi-Fi, mais le système actuel ne permet pas une gestion centralisée ni une automatisation des flux.

Cet inventaire met en évidence les limites de l'existant et la nécessité de mettre en place une solution modernisée, capable de centraliser les données et de supporter un développement structuré avec pipeline **CI/CD** et pratiques **DevOps** .

4.2. Critique de l'existant

Le système actuel repose essentiellement sur des **outils bureautiques locaux** , principalement Microsoft Excel, pour la gestion des dossiers médicaux, le suivi des services et la gestion des stocks. Bien que fonctionnel pour des tâches simples, ce système présente des limites notables pour un fonctionnement efficace et moderne.

Tout d'abord, l'absence de **centralisation des données** entraîne des doublons et ralentit le traitement des informations. Chaque service conserve ses fichiers localement, ce qui complique la consolidation des données et limite la collaboration entre les différents services.

Ensuite, le système **ne permet pas l'automatisation des processus** ni un suivi en temps réel. Toutes les opérations restent manuelles, ce qui peut ralentir certaines tâches et ne facilite pas l'évolution vers une organisation plus efficace.

Enfin, l'infrastructure actuelle ne supporte pas l'intégration d'un **processus DevOps** , tel qu'une pipeline CI/CD pour automatiser le développement, les tests et le déploiement de l'application. L'absence de mécanismes automatisés limite la réactivité et la continuité des mises à jour, éléments essentiels pour un système moderne et évolutif.

En conclusion, le système existant, bien qu'utile pour des tâches basiques, **nécessite une modernisation technologique** . L'introduction d'une solution intégrée, centralisée et automatisée permettra d'optimiser la gestion des services médicaux et de mettre en place un processus de développement structuré et sécurisé conforme aux pratiques **DevOps** et **CI/CD** .

4.3. Proposition des solutions

Suite à l'analyse et à la critique de l'existant, il est apparu nécessaire de moderniser le système de gestion des services médicaux. L'objectif principal est de **passer d'un système manuel, local et non centralisé à une solution automatisée, centralisée et évolutive**, permettant d'optimiser la gestion des informations et d'introduire un processus de développement structuré avec **DevOps et CI/CD**.

4.3.1. Centralisation et modernisation des données

La première solution consiste à mettre en place une **base de données centralisée MySQL**, accessible par tous les services via une application web. Cela permet de :

- **Centraliser les informations** pour éviter les doublons et les incohérences.
- **Accéder aux données en temps réel** depuis n'importe quel poste connecté au réseau.
- **Sécuriser les informations sensibles**, notamment les dossiers médicaux et les stocks pharmaceutiques

4.3.2. Développement structuré de l'application

L'application est développée avec **Node.js pour le backend** et **React.js pour le frontend**, offrant une interface moderne et réactive. Cette solution permet de :

- Développer les fonctionnalités de manière modulable et évolutive.
- Faciliter la maintenance et l'évolution de l'application grâce à un code structuré et standardisé.

4.3.3. Mise en place d'une pipeline CI/CD

L'un des axes majeurs de la modernisation repose sur l'intégration d'une **pipeline CI/CD** via **GitHub Actions et Docker**. Cette approche permet :

- **Validation automatique du code** à chaque modification (lint, tests unitaires avec Jest).
- **Construction et déploiement automatique des conteneurs Docker** sur le serveur VPS.
- **Livraison continue des fonctionnalités en production** sans interruption, assurant un déploiement fiable et rapide.

4.3.4. Automatisation et conteneurisation

L'utilisation de **Docker** et **Docker Compose** assure que l'application soit :

- **Portables et isolées**, avec un environnement identique en développement, test et production.
- **Faciles à déployer et à maintenir**, grâce à des conteneurs standardisés.

En résumé, la solution proposée combine **modernisation des outils, développement structuré, centralisation des données et automatisation DevOps**. Cette approche permet non seulement d'améliorer l'efficacité des services médicaux, mais aussi d'assurer un développement et un déploiement continus, fiables et sécurisés.

4.4. Conception avant la réalisation de l'application

Avant d'entamer la phase de développement, il est essentiel de définir les principes et les choix techniques qui orienteront la construction de l'application. Cette étape de conception permet d'organiser les idées, de structurer le système et de garantir que la solution développée répondra efficacement aux besoins identifiés lors de l'analyse de l'existant. Dans cette optique, plusieurs éléments ont été déterminés : la méthode de modélisation, le langage de programmation, le système de gestion de base de données ainsi que les outils de conception utilisés.

4.4.1. Méthode de modélisation (UML).

Pour la phase de conception, la méthode de modélisation adoptée est **UML (Unified Modeling Language)**. Il s'agit d'un langage de modélisation standard largement utilisé dans les projets informatiques pour représenter, de manière claire et structurée, les différentes composantes d'un système.

L'utilisation d'UML permet de visualiser l'architecture globale de l'application, de définir les interactions entre les acteurs et les fonctionnalités, ainsi que de décrire les structures et comportements internes du système. À travers des diagrammes tels que le diagramme de cas d'utilisation, le diagramme de classes ou encore les diagrammes de séquence, UML facilite la communication entre les parties prenantes et constitue un support solide pour assurer la cohérence entre les besoins exprimés et la solution développée.

Cette approche méthodologique garantit une conception rigoureuse et réduit les risques d'ambiguïté durant les étapes de développement et d'intégration.

4.4.2. Langage de programmation

Le développement de l'application s'appuie sur l'écosystème **JavaScript**, choisi pour sa flexibilité, sa rapidité d'exécution et la cohérence qu'il offre entre le frontend et le backend. La couche serveur (backend) est construite avec **Node.js**, une plateforme permettant d'exécuter JavaScript côté serveur grâce à un modèle non bloquant orienté événements. Cette approche facilite la gestion des requêtes simultanées, garantit de bonnes performances et s'intègre naturellement dans une architecture RESTful moderne.

Pour la couche interface utilisateur (frontend), le framework **React.js** a été retenu. Reconnu pour sa capacité à construire des interfaces dynamiques, réactives et modulaires, React permet une meilleure structuration du code à travers des composants réutilisables, tout en améliorant l'expérience utilisateur grâce à son mécanisme de Virtual DOM.

Afin d'assurer une présentation graphique simple, cohérente et responsive, le framework CSS **Bootstrap** a été utilisé. Celui-ci fournit une base solide de composants visuels standardisés, permettant d'accélérer la conception de l'interface tout en respectant les exigences ergonomiques du projet.

Ce choix technologique Node.js pour le backend, React pour le frontend et Bootstrap pour le style constitue une architecture moderne, scalable et adaptée à une démarche DevOps orientée automatisation et déploiement continu.

4.4.3. Système de gestion de base de données

Le système de gestion de base de données constitue un élément central dans la conception d'une application de gestion médicale, car il assure la structuration, la sécurité et la pérennité des informations manipulées. Dans le cadre de ce projet, le choix s'est orienté vers **MySQL**, un SGBDR (Système de Gestion de Base de Données Relationnel) largement utilisé dans les environnements professionnels.

MySQL se distingue par sa **performance**, sa **robustesse** et sa **capacité à gérer un volume important de données** tout en garantissant une forte cohérence transactionnelle. Ces caractéristiques sont particulièrement adaptées à un système manipulant des données sensibles

telles que les dossiers médicaux, les historiques des services, les informations des patients, ou encore la gestion de stock pharmaceutique.

Sur le plan méthodologique, MySQL facilite :

- **La modélisation relationnelle** grâce à des structures claires (tables, clés primaires, clés étrangères).
- **Le respect des règles d'intégrité** pour éviter les incohérences ou les doublons.
- **L'optimisation des requêtes** pour offrir un accès rapide aux informations.

D'un point de vue DevOps, MySQL présente également plusieurs avantages significatifs. Il s'intègre facilement dans un environnement **conteneurisé avec Docker**, permettant la création d'instances de base de données reproductibles, isolées et rapidement déployables. Cette approche améliore la portabilité du système et facilite les environnements de développement, de test et de production.

Les configurations sensibles (mot de passe root, nom de la base, ports, etc.) sont externalisées dans des **variables d'environnement (.env)**, respectant ainsi les bonnes pratiques CI/CD et renforçant la sécurité. Le déploiement sur le serveur VPS s'effectue également via des conteneurs, garantissant une cohérence totale entre les environnements.

En somme, l'utilisation de MySQL apporte au projet :

- Une **base solide** pour la gestion fiable et sécurisée des données.
- Une **intégration optimale** dans la chaîne DevOps et le pipeline CI/CD.
- Une **scalabilité et une maintenabilité accrue** pour l'évolution future de l'application.

4.4.4. Outils de conception

La conception d'une application moderne nécessite l'utilisation d'outils adaptés permettant de modéliser clairement les besoins, les processus et l'architecture du système. Dans ce projet, l'outil retenu est **Visual Paradigm**, une plateforme complète et largement utilisée dans le cadre de la modélisation UML et de l'ingénierie logicielle.

Visual Paradigm offre un ensemble de fonctionnalités essentielles pour assurer une conception cohérente, structurée et conforme aux standards internationaux :

- **Création de diagrammes UML** (cas d'utilisation, séquences, classes, activités, etc.) indispensables pour représenter graphiquement le fonctionnement de l'application et les interactions entre les différents composants.
- **Support du cycle de développement** : l'outil facilite la transition entre la phase de conception et la phase de réalisation, en transformant les diagrammes en documentation claire et exploitable pour le développement.
- **Gestion centralisée des modèles** : Visual Paradigm permet une organisation rigoureuse des différentes vues du système, ce qui garantit une meilleure compréhension de l'architecture globale.
- **Cohérence entre les artefacts** : les mises à jour effectuées sur un diagramme se répercutent sur les autres modèles liés, assurant ainsi une homogénéité des représentations.

Ainsi, l'utilisation de Visual Paradigm contribue à garantir une conception méthodique et compréhensible, favorisant un développement fluide, une maintenance facilitée et une évolution maîtrisée du système.

4.5. Conception du système

4.5.1. Diagramme de cas d'utilisation global

La conception du système repose sur une approche orientée **objet et utilisateur**, visant à identifier les interactions principales entre les **acteurs du système** et les **fonctionnalités proposées**.

Le diagramme de cas d'utilisation global ci-dessous présente une **vue d'ensemble des interactions** entre les différents utilisateurs et le système. Il permet de visualiser de manière synthétique les services offerts et les responsabilités de chaque acteur.

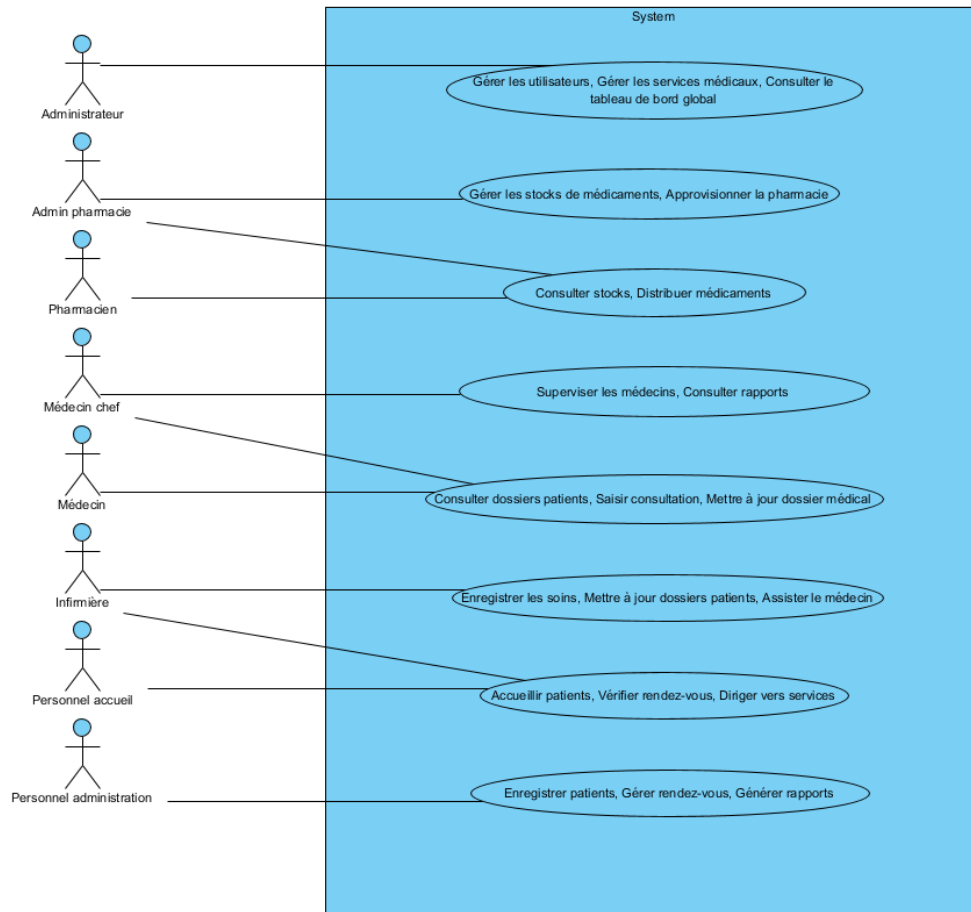


Figure 4 : Diagramme de cas d'utilisation global

Chapitre 5: Revue de littérature

La revue de littérature a pour objectif de présenter les fondements théoriques, conceptuels et technologiques qui soutiennent la modernisation du système d'information réalisée dans ce projet. Elle permet d'examiner les approches contemporaines du développement logiciel, notamment l'intégration de l'approche **DevOps**, l'implémentation de pipelines **CI/CD**, l'utilisation de la **conteneurisation** ainsi que les technologies Web modernes. Cette revue offre ainsi un cadre de compréhension solide reliant les concepts académiques aux solutions pratiques adoptées pour transformer un système bureautique traditionnel en une application moderne, évolutive et automatisée.

5.1. Modernisation des systèmes d'information

La modernisation des systèmes d'information constitue aujourd'hui un enjeu stratégique majeur pour les organisations souhaitant améliorer leur efficacité opérationnelle, leur capacité d'innovation et leur résilience face aux exigences numériques croissantes. Dans un contexte marqué par l'évolution rapide des technologies, la montée en puissance du cloud computing et l'importance accordée à la sécurité informatique, les entreprises sont amenées à repenser en profondeur leurs architectures, leurs processus de développement et leurs outils techniques.

La modernisation ne se limite pas à une simple mise à jour des technologies existantes ; elle implique une transformation globale visant à rendre les systèmes plus modulaires, automatisés et scalables. Cette démarche repose souvent sur l'adoption de pratiques émergentes telles que DevOps, l'intégration et le déploiement continu (CI/CD), la conteneurisation des applications ou encore l'orchestration d'infrastructures virtualisées. Ces approches permettent de réduire les temps de mise en production, d'améliorer la qualité logicielle et d'assurer une meilleure fiabilité des services informatiques.

Dans le domaine des services médicaux interentreprises, cette modernisation revêt une importance particulière. Les systèmes historiques sont souvent monolithiques, difficilement maintenables et peu adaptés aux besoins actuels en matière de traçabilité, de performance et de sécurité des données sensibles. L'évolution vers des solutions plus modernes, fondées sur des technologies web, des environnements virtualisés et des pipelines automatisés, facilite la gestion des opérations quotidiennes, renforce la protection des informations médicales et offre une meilleure capacité d'adaptation aux changements organisationnels.

Ainsi, la modernisation des systèmes d'information s'impose comme un levier essentiel pour optimiser les processus internes, soutenir la transformation digitale des organisations et garantir la pérennité de leurs activités dans un environnement technologique en constante mutation.

5.2. Approche DevOps

5.2.1. Définition

Le terme *DevOps* désigne une approche organisationnelle qui vise à renforcer la collaboration entre les équipes de développement (Dev) et d'exploitation (Ops). Elle repose sur une culture de communication continue, d'automatisation des processus et d'intégration régulière du code, afin d'accélérer le cycle de livraison des applications tout en garantissant leur fiabilité et leur performance. DevOps n'est donc pas uniquement un ensemble d'outils, mais un ensemble de pratiques et de principes facilitant une modernisation progressive et maîtrisée des systèmes d'information.

5.2.2. Avantages de l'approche DevOps

L'adoption du DevOps présente de nombreux bénéfices pour les organisations souhaitant moderniser leurs systèmes informatiques :

- **Réduction du temps de mise en production** grâce à l'automatisation des tests, des intégrations et des déploiements.
- **Amélioration de la qualité logicielle** par des retours rapides, des tests systématiques et une détection précoce des anomalies.
- **Collaboration renforcée** entre développeurs et administrateurs système, réduisant les malentendus et les délais liés aux validations manuelles.
- **Stabilité accrue des environnements** grâce à la standardisation des configurations et à la continuité entre développement et production.
- **Capacité d'innovation plus élevée**, car les équipes passent moins de temps à résoudre des incidents récurrents et peuvent se concentrer sur la valeur métier.

5.2.3. Cycle de vie DevOps

Le cycle de vie DevOps représente un processus continu et itératif, permettant d'intégrer, de tester et de déployer les applications de manière rapide, fiable et reproductible. Dans le cadre de ce projet, ce cycle a été appliqué de manière concrète afin de moderniser le système d'information médical tout en assurant la qualité et la disponibilité du service.

Le cycle comprend les étapes suivantes :

❖ **Planification :**

Chaque fonctionnalité ou modification est d'abord planifiée en fonction des besoins des utilisateurs et des objectifs du système. Cette étape permet de définir les tâches, de hiérarchiser les priorités et de préparer l'environnement de développement.

❖ **Développement :**

Le code est développé en utilisant **Node.js** pour le backend et **React.js** pour le frontend, en suivant les bonnes pratiques de modularité et de maintenabilité. Les développeurs utilisent **Git** pour la gestion des versions et **GitHub** comme dépôt central afin de suivre l'évolution du code source.

❖ **Tests automatisés :**

Chaque modification est validée automatiquement par des tests unitaires et d'intégration. Cette automatisation permet de détecter rapidement les erreurs et de garantir la stabilité de l'application avant toute phase de déploiement.

❖ **Build et conteneurisation :**

Les applications sont empaquetées dans des conteneurs via **Docker**, avec une orchestration simplifiée par **Docker Compose**. Les images Docker sont ensuite publiées dans **GHCR (GitHub Container Registry)**, ce qui assure leur traçabilité et leur portabilité entre différents environnements.

❖ **Déploiement sur le VPS :**

Les conteneurs sont déployés sur le serveur VPS, avec **Nginx** utilisé comme serveur web pour gérer les requêtes et sécuriser l'accès à l'application. Cette étape garantit que l'application fonctionne correctement dans un environnement proche de la production.

❖ **Boucle continue et retour d'expérience :**

Le cycle est répété à chaque nouvelle fonctionnalité ou correction, permettant une amélioration continue du système. Cette approche assure non seulement la qualité du code mais aussi la rapidité de livraison, la flexibilité et la fiabilité de l'application pour les utilisateurs.

L'ensemble de ces étapes forme un **pipeline DevOps cohérent**, intégrant planification, développement, tests, build et déploiement automatisé. Cette méthodologie assure une livraison continue, réduit les risques d'erreurs humaines et facilite la maintenance et l'évolution future du système.

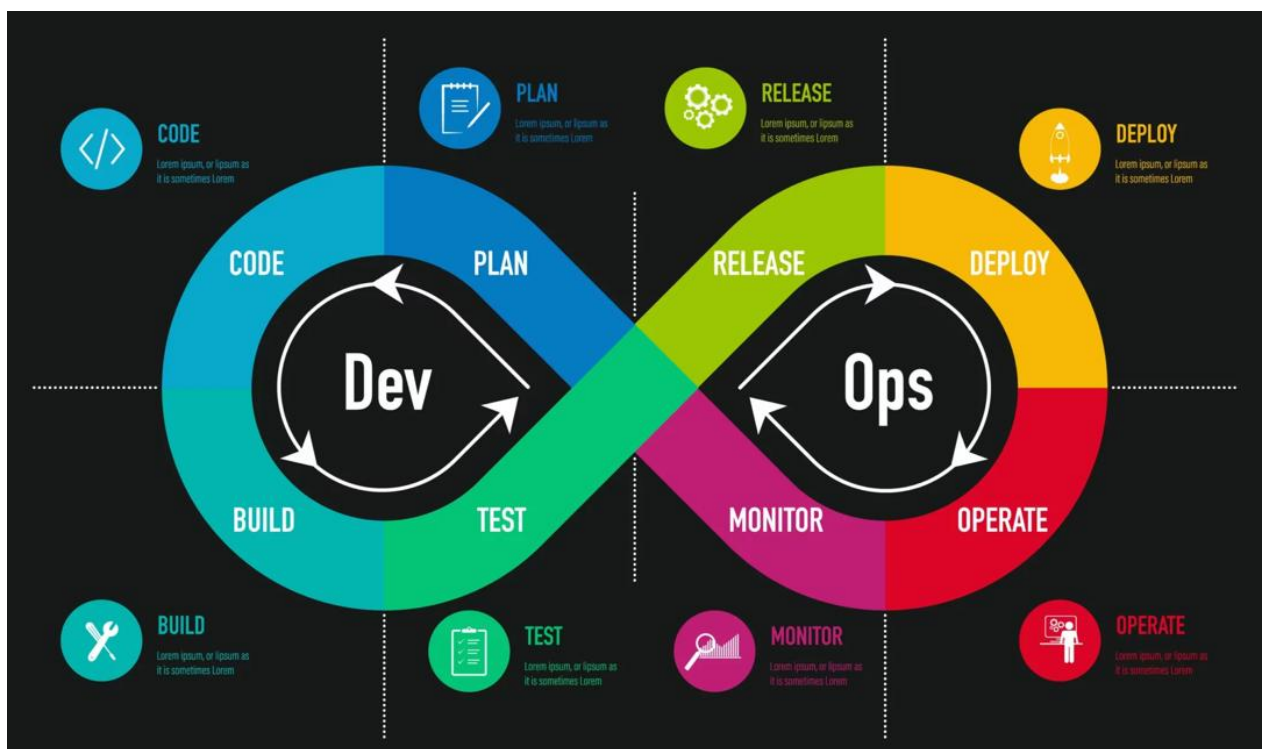


Figure 5 : Cycle de vie DevOps

Chapitre 6: Choix des outils

Ce chapitre présente et justifie de manière détaillée l'ensemble des outils technologiques sélectionnés dans le cadre de la modernisation du Système de Gestion des Services Médicaux Interentreprises. L'objectif est de mettre en évidence la pertinence de chaque choix technologique au regard des exigences fonctionnelles et techniques du projet. Les solutions retenues répondent non seulement aux besoins de performance, de sécurité et de fiabilité, mais également aux impératifs de maintenabilité, de scalabilité et d'automatisation imposés par une démarche DevOps moderne. Ainsi, chaque outil contribue à la constitution d'une architecture cohérente, durable et optimale pour soutenir efficacement les processus métiers et garantir un déploiement continu maîtrisé.

Ce chapitre présente et justifie l'ensemble des outils utilisés dans la modernisation du Système de Gestion des Services Médicaux Interentreprises. Le choix de ces technologies repose sur des critères de performance, de sécurité, de fiabilité, de maintenabilité et d'intégration dans une approche DevOps moderne.

6.1. Serveur VPS – OVHCloud

Le Serveur VPS constitue la pierre angulaire de l'infrastructure déployée dans le cadre de la modernisation du système. Il représente l'environnement principal dans lequel les services applicatifs, les conteneurs Docker et les outils de sécurité sont exécutés. Le choix d'un VPS OVHCloud répond à des exigences de fiabilité, de performance, d'évolutivité et de maîtrise complète de l'environnement système.

6.1.1. Justification du choix du fournisseur OVHCloud

OVHCloud est reconnu pour la qualité de ses infrastructures, la stabilité de ses services et la transparence de son écosystème. Les éléments suivants motivent ce choix :

- **Haute disponibilité et fiabilité du réseau.**
- **Infrastructure performante** basée sur des ressources dédiées.
- **Localisation des centres de données conforme aux normes internationales.**
- **Support efficace et documentation complète.**

6.1.2. Performance et caractéristiques techniques du VPS

Le VPS sélectionné offre des caractéristiques adaptées aux besoins du système modernisé :

- CPU et RAM suffisants pour supporter plusieurs conteneurs simultanément.
- Stockage SSD/NVMe pour des temps de réponse optimisés.
- Système d'exploitation Linux (Ubuntu/Debian).

6.1.3. Sécurité et accès au serveur

La sécurité constitue un élément essentiel dans la gestion d'un serveur VPS. L'accès s'effectue exclusivement via une **clé SSH**, garantissant une authentification forte et limitant les tentatives d'accès non autorisées. Le VPS permet également une configuration fine du pare-feu afin de contrôler le trafic réseau de manière granulaire. Grâce à Docker, chaque service est isolé dans son propre conteneur, réduisant les risques de propagation en cas d'incident. Enfin, la mise en place de certificats SSL assure la sécurisation des échanges entre les utilisateurs et les services exposés.

6.1.4. Compatibilité avec Docker et la CI/CD

Le VPS offre un environnement parfaitement compatible avec Docker, permettant de déployer les différents services de manière standardisée, portable et cohérente. Cette compatibilité s'étend également aux pipelines GitHub Actions, qui automatisent le processus de construction, de test, de création d'images Docker, puis de déploiement sur le serveur via SSH. Grâce à cette intégration fluide, la chaîne CI/CD garantit une livraison continue fiable, rapide et totalement reproductible.

6.1.5. Scalabilité et maîtrise de l'infrastructure

Le VPS se distingue par sa capacité d'adaptation aux besoins évolutifs du système. Il permet d'ajuster dynamiquement les ressources (CPU, mémoire, stockage) en fonction des charges de travail ou de l'évolution du projet. L'administrateur bénéficie d'un contrôle total sur l'architecture, la configuration réseau et l'orchestration des services, offrant une flexibilité essentielle pour optimiser la performance et la disponibilité. Cette maîtrise complète de l'infrastructure permet également d'intégrer facilement de nouveaux services ou d'améliorer les composants existants.

6.2. Outils de conteneurisation

La conteneurisation joue un rôle central dans la modernisation et l'automatisation des applications. Elle permet d'assurer la portabilité, la cohérence et la reproductibilité des environnements, tout en facilitant le déploiement continu et la maintenance. Dans le cadre de ce projet, plusieurs solutions ont été analysées, et **Docker** accompagné de **Docker Compose** a été retenu pour répondre aux exigences fonctionnelles et techniques du système.

6.2.1. Docker

Docker et Podman sont deux solutions de conteneurisation populaires dans les entreprises, chacune ayant ses avantages et ses inconvénients. HR IT-Consulting utilise déjà Docker pour la conteneurisation de microservices. Cependant, pour déterminer quel outil de conteneurisation sera utilisé pour les applications serveurs dans le cadre de ce projet, il est crucial d'évaluer ces deux solutions sur plusieurs aspects clés.

Tableau 9 : Comparaison Docker et Podman

Critère	Docker	Podman
Architecture	Nécessite un démon central (Docker Daemon)	Sans démon, fonctionne de manière daemonless
Popularité	Très largement adopté, support communautaire élevé	Adoption croissante mais moins répandue
Intégration CI/CD	Intégration native avec GitHub Actions, Docker Hub et autres outils DevOps	Intégration possible mais moins répandue, nécessite ajustements
Complexité	Facile à prendre en main et à configurer	Plus complexe pour certains workflows DevOps traditionnels

Sécurité	Isolation standard, support des namespaces Linux	Isolation plus poussée, modèle sans démon réduit certains risques
-----------------	--	---

Docker est un outil de conteneurisation qui encapsule une application et toutes ses dépendances dans un conteneur léger et isolé. Cette approche garantit une exécution identique entre les différents environnements, du développement jusqu'à la production.

Le choix de Docker se justifie par plusieurs raisons :

- **Simplicité et maturité** : Docker est l'outil de conteneurisation le plus répandu, avec un vaste écosystème et une documentation très complète, facilitant ainsi la prise en main et la résolution de problèmes techniques.
- **Intégration DevOps** : Docker s'intègre de manière transparente avec les pipelines CI/CD, notamment GitHub Actions, ce qui permet d'automatiser les builds, les tests, et les déploiements.
- **Gestion des versions et portabilité** : Grâce aux images Docker, il est possible de standardiser les environnements et de garantir que le code fonctionne exactement de la même manière sur différents serveurs.
- **Support communautaire** : Une communauté active assure un suivi continu, des mises à jour régulières et de nombreux tutoriels pour la résolution de problèmes.

Docker est retenu pour ce projet car il combine fiabilité, intégration native avec l'écosystème DevOps, simplicité d'utilisation et portabilité des applications, tandis que Podman, bien que sécuritaire et intéressant pour des environnements spécialisés, nécessiterait des ajustements supplémentaires et une courbe d'apprentissage plus importante.

6.2.2. Docker Compose

Docker Compose permet d'orchestrer plusieurs conteneurs de manière coordonnée via un fichier unique (docker-compose.yml). Cet outil facilite la définition des services, des réseaux et des

volumes, permettant un démarrage rapide et reproductible de l'ensemble de l'infrastructure applicative.

Le choix de Docker Compose est justifié par :

- **Simplicité pour les projets multi-services** : coordination facile entre backend, frontend, base de données et proxy inverse.
- **Reproductibilité** : possibilité de lancer tout l'environnement avec une seule commande.
- **Maintenance et évolutivité** : configuration centralisée facilitant les mises à jour, les modifications de service ou l'ajout de nouveaux conteneurs.
- **Compatibilité avec Docker** : Docker Compose s'intègre parfaitement à Docker et aux pipelines CI/CD automatisés, permettant ainsi un déploiement cohérent et fiable sur le serveur VPS.

Ainsi, l'association Docker + Docker Compose constitue un choix stratégique, garantissant une conteneurisation efficace, une orchestration simplifiée et une intégration fluide dans l'infrastructure DevOps du projet.

6.3. Répertoire d'images Docker – GitHub Container Registry (GHCR)

Le stockage et la gestion des images Docker constituent un élément fondamental pour assurer la cohérence, la portabilité et la reproductibilité des applications tout au long du cycle de vie du projet. Dans ce contexte, le choix du registre de conteneurs est stratégique pour garantir une intégration fluide avec les processus de développement et de déploiement. Pour ce projet, **GitHub Container Registry (GHCR)** a été retenu comme solution principale pour héberger et gérer les images Docker.

6.3.1. Présentation du GHCR

GitHub Container Registry est un service proposé par GitHub pour l'hébergement et la gestion des images de conteneurs. Il offre des fonctionnalités avancées permettant de gérer à la fois des images publiques et privées. GHCR est étroitement intégré à l'écosystème GitHub, permettant aux équipes de développement de bénéficier d'un workflow fluide, depuis le code source jusqu'au déploiement en production. L'utilisation de GHCR permet également un contrôle précis des versions des images, ce qui est essentiel pour assurer la traçabilité et la stabilité des déploiements.

6.3.2. Justification du choix

Le choix de GHCR est motivé par plusieurs avantages significatifs :

- **Intégration complète avec GitHub Actions** : GHCR s'intègre de manière native avec GitHub Actions, facilitant l'automatisation des builds, des tests et des déploiements d'images Docker.
- **Sécurité renforcée** : le registre supporte l'authentification via token personnel et offre un contrôle strict des accès aux images, garantissant la protection des artefacts critiques du projet.
- **Gestion précise des versions** : chaque image peut être taguée et versionnée, ce qui permet de déployer exactement la version souhaitée sur l'infrastructure VPS, tout en conservant un historique complet des modifications.
- **Fiabilité et performance** : en s'appuyant sur l'infrastructure GitHub, GHCR assure une haute disponibilité et un accès rapide aux images, même pour des projets distribués.
- **Flexibilité d'utilisation** : possibilité de créer des images publiques ou privées selon les besoins, ce qui offre une souplesse adaptée aux différents scénarios de déploiement.

6.3.3. Comparaison avec d'autres registres Docker

Afin de justifier le choix de GHCR, il est pertinent de comparer ses fonctionnalités avec d'autres solutions populaires de registres Docker, comme Docker Hub, pour mettre en évidence ses avantages et sa pertinence dans le cadre de ce projet.

Tableau 10 : Comparaison GHCR et Docker Hub

Critère	GHCR	Docker Hub
Intégration CI/CD	Intégration native avec GitHub Actions, facile à automatiser	Intégration possible mais nécessite une configuration supplémentaire et parfois complexe
Sécurité	Authentification via token personnel, contrôle fin des dépôts privés	Authentification via compte Docker, certaines limitations sur les dépôts privés gratuits

Gestion des versions	Versionnement précis avec support complet des tags multiples, traçabilité assurée	Gestion des tags standard, moins adaptée aux projets complexes nécessitant un suivi détaillé
Disponibilité	Haute disponibilité grâce à l'infrastructure GitHub	Large disponibilité, mais dépend de l'infrastructure Docker Hub et de la connectivité réseau

GHCR a été choisi pour ce projet en raison de son intégration directe avec les workflows GitHub Actions, sa sécurité renforcée, sa flexibilité et sa capacité à gérer efficacement les images Docker utilisées dans les déploiements automatisés sur le serveur VPS. Son adoption permet de garantir une continuité de service, une traçabilité complète des versions et une gestion optimisée des artefacts logiciels.

6.4. Intégration Continue (CI) & Déploiement Continu (CD)

L'automatisation des processus de développement, de test et de déploiement est au cœur de la démarche DevOps. L'intégration continue (CI) et le déploiement continu (CD) permettent de garantir que chaque modification du code est testée, validée et déployée de manière sécurisée et fiable, réduisant ainsi les erreurs humaines et les interruptions de service. Dans ce projet, **GitHub Actions** a été choisi comme outil principal pour orchestrer le pipeline CI/CD en raison de son intégration native avec GitHub, sa flexibilité et sa simplicité d'utilisation.

6.4.1. Intégration Continue (CI)

L'intégration continue consiste à vérifier automatiquement chaque modification apportée au code source. GitHub Actions permet de :

- **Exécuter les tests unitaires et d'intégration** à chaque push ou pull request, garantissant la stabilité et la qualité du code.
- **Construire automatiquement les images Docker** pour le backend, le frontend et la base de données, assurant que les conteneurs sont toujours à jour et prêts pour le déploiement.
- **Analyser la qualité du code** via des outils comme linters ou SonarCloud intégrés dans le workflow, permettant de détecter les erreurs ou incohérences rapidement.

- **Gérer les branches et la validation des pull requests**, assurant que seules les modifications conformes et validées sont fusionnées dans la branche principale.

6.4.2. Déploiement Continu (CD)

Le déploiement continu permet de publier automatiquement les changements validés sur l'infrastructure de production. GitHub Actions offre :

- **Push automatique des images vers GHCR**, garantissant que les conteneurs utilisés sont toujours synchronisés avec la version du code validée.
- **Connexion sécurisée au serveur VPS via SSH**, permettant d'exécuter les commandes de déploiement à distance de manière sécurisée.
- **Mise à jour et redémarrage des conteneurs Docker** sans interruption de service, permettant de maintenir la disponibilité de l'application.
- **Automatisation complète du pipeline**, réduisant les risques d'erreurs manuelles et améliorant la rapidité des déploiements.

6.4.3. Comparaison avec d'autres outils CI/CD

Afin de justifier le choix de GitHub Actions, une comparaison avec d'autres solutions populaires comme Jenkins et GitLab CI/CD est présentée :

Tableau 11 : Comparaison Github Actions avec Jenkins et Gitlab CI/CD

Critère	GitHub Actions	Jenkins	GitLab CI/CD
Intégration avec Git	Native, immédiate	Nécessite plugin ou configuration	Native avec GitLab repos
Configuration	YAML, simple à versionner	GUI complexe et scripts	YAML similaire à GitHub Actions
Maintenance	Faible, intégré	Plus lourde, serveur dédié	Moyenne, intégré à GitLab
Coût	Inclus dans GitHub, gratuit pour projets publics	Serveur dédié, licence possible	Inclus dans GitLab, gratuit pour projets publics

Facilité d'utilisation	Très intuitive, bonne documentation	Complexe pour débutants	Bonne, mais moins populaire que GitHub Actions
-------------------------------	-------------------------------------	-------------------------	--

6.4.4. Justification du choix de GitHub Actions

GitHub Actions a été retenu pour plusieurs raisons :

- **Intégration native avec le dépôt GitHub**, simplifiant la configuration et la maintenance du pipeline CI/CD.
- **Flexibilité et personnalisation**, permettant de créer des workflows adaptés aux besoins spécifiques du projet.
- **Support multi-plateforme et multi-environnement**, compatible avec Windows, Linux et conteneurs Docker.
- **Communauté active et documentation abondante**, facilitant la résolution de problèmes et l'évolution des pipelines.
- **Coût et accessibilité**, inclus gratuitement avec les dépôts GitHub pour les projets de taille moyenne.

GitHub Actions a été choisi pour sa simplicité, son intégration native avec GitHub, sa flexibilité et sa capacité à automatiser entièrement le pipeline CI/CD. Il permet de garantir une livraison rapide, sécurisée et reproductible des applications conteneurisées sur le VPS.

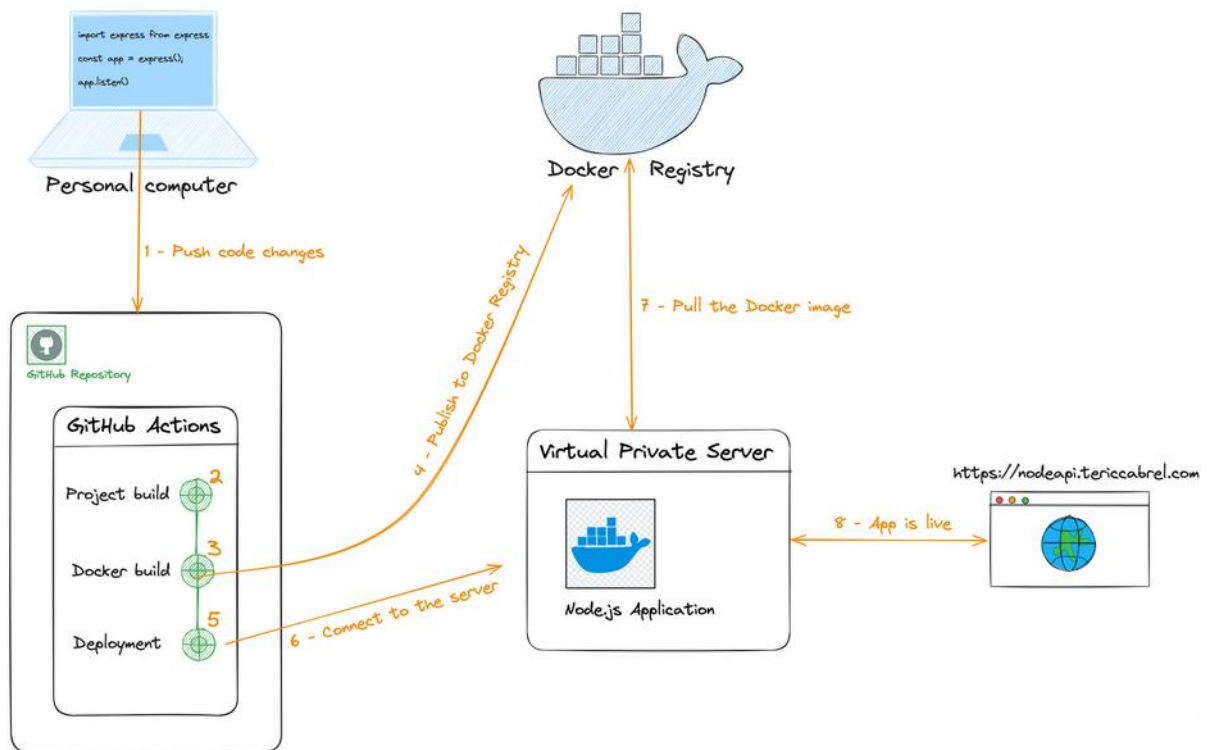


Figure 6 : Illustration du pipeline CI/CD Github action, docker

6.5. Outils de développement

Le choix des outils de développement constitue une étape stratégique dans la modernisation du Système de Gestion des Services Médicaux Interentreprises. Ces outils ont été sélectionnés en fonction de leur compatibilité avec les technologies modernes, leur performance, leur capacité à s'intégrer dans un pipeline DevOps, leur sécurité et leur soutien par une communauté active. L'objectif principal est de garantir la création d'une application robuste, maintenable, évolutive et capable de répondre efficacement aux besoins des utilisateurs finaux.

6.5.1. Backend – Node.js

Node.js est utilisé pour développer le backend en raison de sa rapidité, de sa scalabilité et de sa large adoption dans l'industrie. Ses principaux avantages sont :

- **Performances élevées** grâce à son moteur V8 et sa gestion non bloquante des entrées/sorties, permettant de traiter un grand nombre de requêtes simultanément sans compromettre la réactivité du système.

- **Écosystème riche** avec un large nombre de modules disponibles via npm, facilitant l'ajout de nouvelles fonctionnalités et accélérant le développement.
- **Compatibilité avec Docker et CI/CD**, ce qui permet d'intégrer le backend de manière fluide dans le pipeline d'automatisation et de garantir une livraison rapide et sécurisée.
- **Large communauté et support à long terme**, offrant un suivi continu des mises à jour et une documentation complète pour résoudre rapidement les problèmes.
- **Scalabilité et flexibilité** permettant d'adapter facilement l'architecture backend aux besoins futurs, notamment pour gérer une augmentation du nombre d'utilisateurs ou des fonctionnalités supplémentaires.

6.5.2. Frontend – React.js

React.js a été choisi pour le développement du frontend pour sa capacité à construire des interfaces utilisateurs dynamiques, réactives et modulaires. Les avantages sont multiples :

- **Composants réutilisables**, permettant de réduire la redondance du code et de simplifier la maintenance sur le long terme.
- **Écosystème riche** (Vite, Redux, Hooks, React Router) facilitant l'intégration avec des outils modernes et l'optimisation des performances.
- **Compatibilité avec les pipelines CI/CD**, garantissant que les mises à jour du frontend sont testées et déployées automatiquement sans intervention manuelle.
- **Communauté active**, offrant un large support technique, des tutoriels et des modules complémentaires pour enrichir l'interface utilisateur et accélérer le développement.
- **Réactivité et expérience utilisateur optimisée**, essentielle pour un système médical où la rapidité d'accès à l'information est cruciale.

6.5.3. Base de données – MySQL

MySQL est utilisé comme système de gestion de base de données relationnelle en raison de sa robustesse, de sa performance et de sa compatibilité avec les conteneurs Docker. Ses principaux avantages incluent :

- **Fiabilité et performance**, adaptées aux applications critiques nécessitant une disponibilité continue et une gestion efficace des données.
- **Compatibilité avec les ORM modernes**, facilitant le développement backend et l'interaction avec les données de manière sécurisée et structurée.
- **Simplicité d'intégration avec Docker**, permettant de déployer rapidement et de manière reproductible la base de données dans l'environnement de production.
- **Large adoption et support communautaire**, assurant un environnement stable, documenté et maintenu à jour.
- **Sécurité et sauvegarde des données**, avec des mécanismes de réplication, de sauvegarde et de restauration adaptés aux besoins d'un système médical interentreprises.

6.5.4. Reverse proxy – Nginx

Nginx est choisi comme reverse proxy pour gérer efficacement le trafic réseau, améliorer la sécurité et optimiser la performance du système. Ses principaux avantages sont :

- **Gestion efficace du routage et des connexions** pour les services frontend et backend, permettant une répartition intelligente de la charge et une meilleure disponibilité des applications.
- **Support du HTTPS et des certificats SSL**, garantissant la sécurité des communications entre les utilisateurs et le serveur, essentielle pour protéger les données médicales sensibles.
- **Performance et caching** pour améliorer la rapidité des réponses et réduire la charge sur le serveur, contribuant à une expérience utilisateur fluide.
- **Compatibilité avec Docker**, permettant de déployer Nginx dans un conteneur isolé, simplifiant la maintenance et les mises à jour.
- **Flexibilité pour l'évolution future**, avec la possibilité de configurer des règles avancées de routage, de load balancing et de redirection selon les besoins du projet.

La combinaison de Node.js, React.js, MySQL et Nginx constitue une architecture moderne, performante, sécurisée et flexible. Ces outils permettent de développer un système cohérent et maintenable, tout en facilitant l'intégration avec les pipelines CI/CD et les conteneurs Docker sur

le VPS. En outre, leur adoption garantit la pérennité de la solution, la scalabilité pour répondre à la croissance des besoins et la possibilité d'intégrer de nouvelles fonctionnalités avec un minimum de complexité. Cette approche assure une modernisation complète du système médical tout en respectant les meilleures pratiques DevOps et les standards industriels actuels.

Le chapitre 6 a présenté les outils sélectionnés pour la modernisation du Système de Gestion des Services Médicaux Interentreprises, en mettant l'accent sur leur performance, sécurité, fiabilité et compatibilité DevOps. L'infrastructure VPS OVHCloud assure un environnement stable et évolutif, tandis que Docker et Docker Compose permettent la conteneurisation et la portabilité des applications. GitHub Container Registry (GHCR) facilite le stockage sécurisé et le versionnage des images, et GitHub Actions automatise l'intégration continue et le déploiement continu, garantissant la qualité et la fiabilité du code. Enfin, Node.js, React.js, MySQL et Nginx forment un ensemble cohérent et performant, assurant une architecture moderne, sécurisée, maintenable et évolutive, capable de répondre efficacement aux besoins des utilisateurs tout en respectant les meilleures pratiques DevOps.

PARTIE III : MISE EN ŒUVRE

Chapitre 7: Réalisation du projet

Ce chapitre présente de manière approfondie la réalisation du projet de modernisation du Système de Gestion des Services Médicaux Interentreprises. Il décrit les étapes de conception, de développement, de configuration et de déploiement de l'application, en mettant particulièrement l'accent sur l'intégration continue et le déploiement continu (CI/CD), ainsi que sur les méthodes, outils et bonnes pratiques adoptés pour garantir la fiabilité, la sécurité, la performance et la maintenabilité du système.

7.1. Préparation de l'environnement

L'installation et la configuration de l'environnement de travail constituent une étape essentielle pour garantir la cohérence, la stabilité et la reproductibilité des développements et des déploiements. Cette phase permet également d'assurer que l'ensemble des outils nécessaires au projet, y compris les pipelines CI/CD, sont correctement configurés et opérationnels.

7.1.1. Serveur VPS OVHCloud

Le serveur VPS OVHCloud constitue l'infrastructure centrale sur laquelle l'ensemble du système sera déployé. Le choix de OVHCloud est motivé par sa fiabilité, sa flexibilité et la possibilité de dimensionner les ressources selon les besoins du projet.

a) Acquisition du serveur

La première étape consiste à sélectionner un VPS OVHCloud avec des caractéristiques techniques adaptées aux besoins du projet. Il est essentiel de choisir un serveur disposant de suffisamment de CPU, de RAM et d'espace de stockage pour supporter les conteneurs Docker ainsi que l'ensemble des services de l'application, garantissant ainsi des performances optimales même en cas de forte charge.

Le choix du système d'exploitation Linux Ubuntu 22.04 LTS est motivé par sa stabilité, sa sécurité, sa large compatibilité avec les outils de développement (Node.js, React.js, MySQL, Nginx) et sa facilité de gestion sur des environnements serveur. Ubuntu LTS offre également un support long terme et une communauté active pour résoudre rapidement tout problème éventuel.

OVHCloud propose des options avancées de sauvegarde et de monitoring, permettant de protéger les données et d'assurer la continuité du service en cas d'incident. Ces fonctionnalités

incluent la sauvegarde automatique du VPS, le monitoring des ressources (CPU, RAM, disque) et des alertes en cas de surcharge ou de dysfonctionnement.

BLACK FRIDAY Votre option de sauvegarde premium offerte

Bénéficiez de la sauvegarde automatique premium* sans surcoût :

- 1 mois offert sans engagement
- 2 mois offerts pour un engagement de 6 mois
- 3 mois offerts pour un engagement de 12 mois

VPS 2026
Choisissez la configuration qui correspond à vos besoins : hébergement web, gaming, VPN, développement et plus encore.

VPS-1	VPS-2	VPS-3	VPS-4	VPS-5	VPS-6
À partir de 3,82 € HT/mois soit 4,58 € TTC/mois Frais d'installation: Offert	À partir de 5,95 € HT/mois soit 7,14 € TTC/mois Frais d'installation: Offert	À partir de 11,90 € HT/mois soit 14,28 € TTC/mois Frais d'installation: Offert	À partir de 21,25 € HT/mois soit 25,50 € TTC/mois Frais d'installation: Offert	À partir de 31,45 € HT/mois soit 37,74 € TTC/mois Frais d'installation: Offert	À partir de 41,65 € HT/mois soit 49,98 € TTC/mois Frais d'installation: Offert
Configurer	Configurer	Configurer	Configurer	Configurer	Configurer
✓ 4 vCores ✓ 8 Go RAM ✓ 75 Go SSD ✓ Sauvegarde automatisée 1 jour ✓ Trafic illimité	✓ 6 vCores ✓ 12 Go RAM ✓ 100 Go SSD NVMe ✓ Sauvegarde automatisée 1 jour ✓ Trafic illimité	✓ 8 vCores ✓ 24 Go RAM ✓ 200 Go SSD NVMe ✓ Sauvegarde automatisée 1 jour ✓ Trafic illimité	✓ 12 vCores ✓ 48 Go RAM ✓ 300 Go SSD NVMe ✓ Sauvegarde automatisée 1 jour ✓ Trafic illimité	✓ 16 vCores ✓ 64 Go RAM ✓ 350 Go SSD NVMe ✓ Sauvegarde automatisée 1 jour ✓ Trafic illimité	✓ 24 vCores ✓ 96 Go RAM ✓ 400 Go SSD NVMe ✓ Sauvegarde automatisée 1 jour ✓ Trafic illimité

Figure 7 : Offre VPS OVHCloud

L'accès au tableau de bord OVHCloud permet une gestion centralisée du VPS. Depuis ce tableau de bord, il est possible de configurer les options du serveur, suivre l'état des ressources en temps réel, redémarrer ou restaurer le serveur, et gérer les snapshots et les sauvegardes. Cela offre une vision complète de l'infrastructure et facilite la prise de décisions rapides et efficaces en termes de gestion et de maintenance.

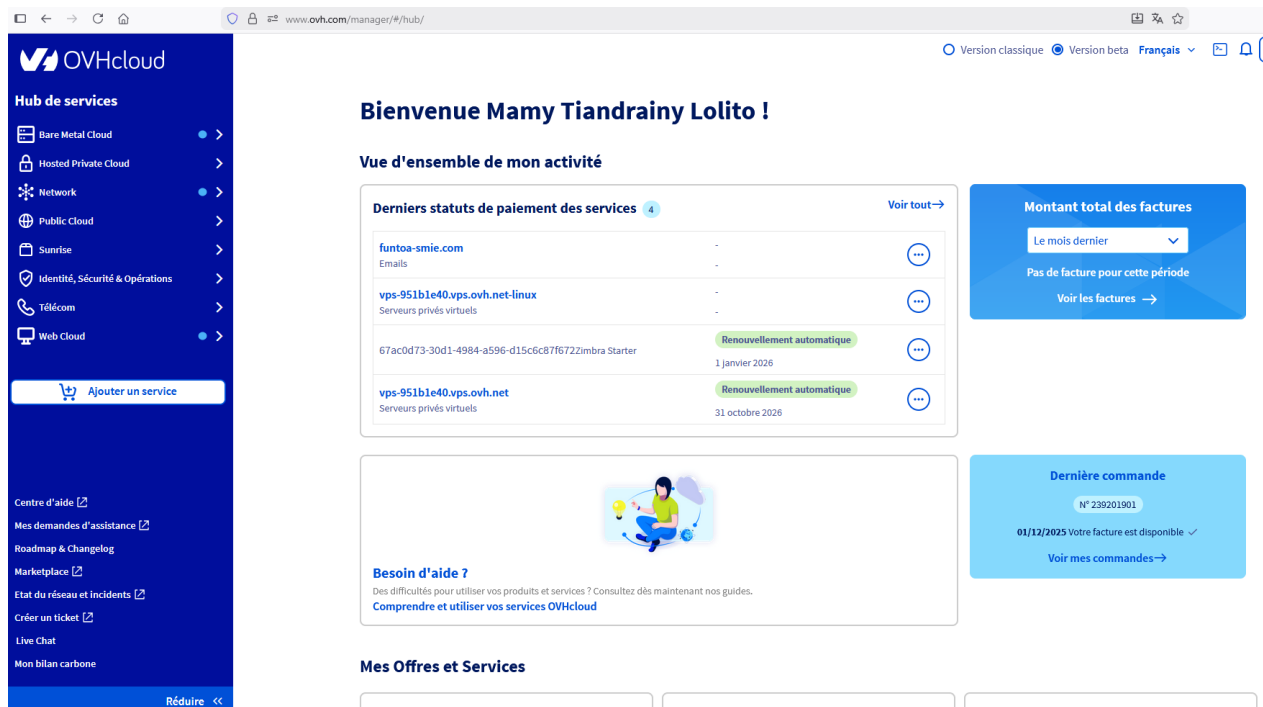


Figure 8 : Tableau de bord OVHCloud

b) Connexion au serveur VPS avec Bitwise SSH Client

La connexion au serveur VPS constitue une étape essentielle pour gérer l'infrastructure et déployer les services nécessaires à l'application. Pour ce projet, nous avons utilisé **Bitwise SSH Client**, un client SSH pour Windows qui offre une interface graphique intuitive et des fonctionnalités avancées pour établir des connexions sécurisées.

La figure montre l'interface du client SSH avant l'établissement de la connexion. On peut y voir les champs de configuration pour l'adresse IP du serveur, le port SSH et le nom d'utilisateur.

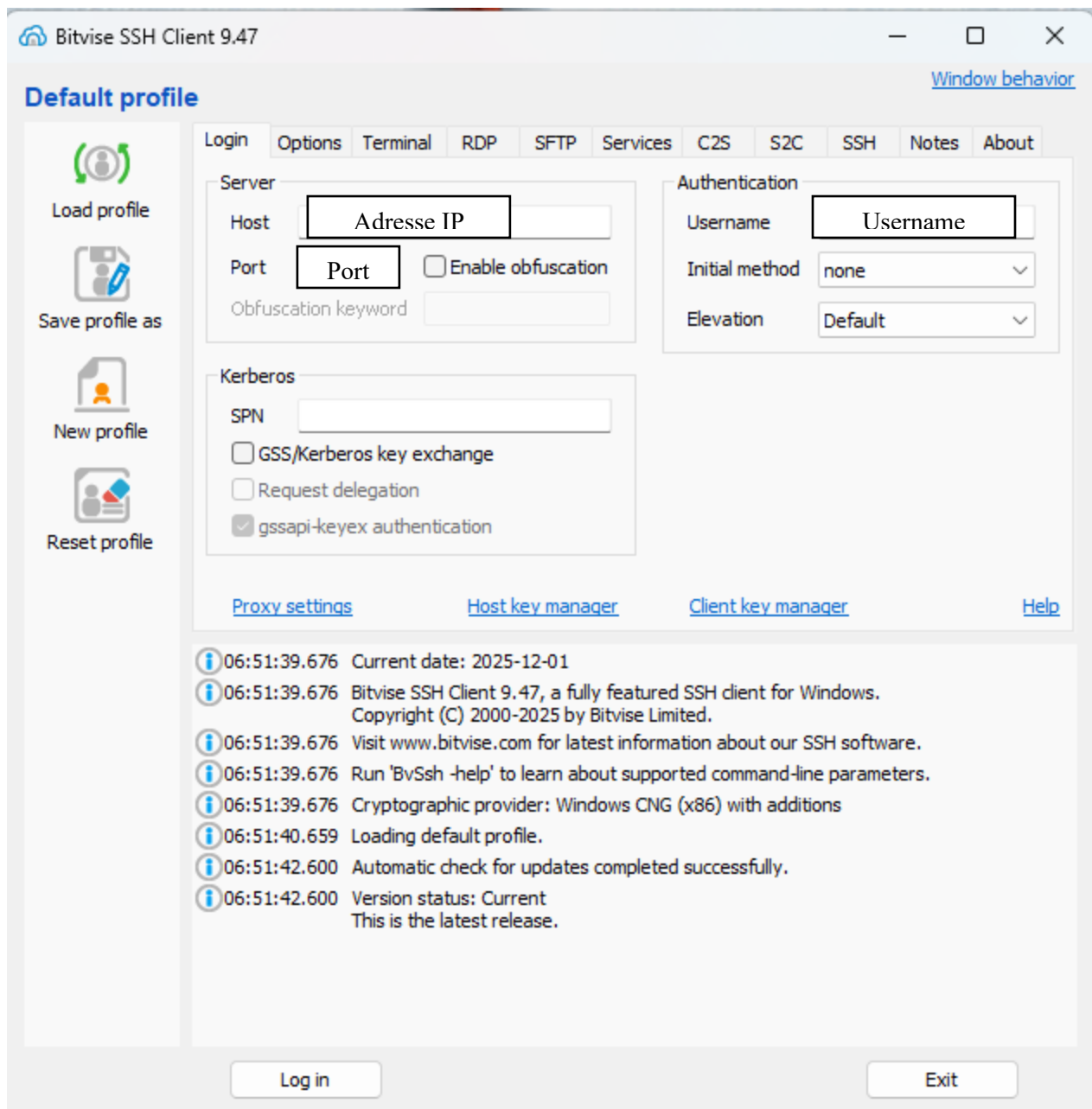


Figure 9 : Bitvise SSH Client non connecté

La figure illustre l'état du client après la connexion réussie au VPS. Les informations de session et l'accès aux fichiers distants sont visibles, indiquant que la connexion est sécurisée et opérationnelle.

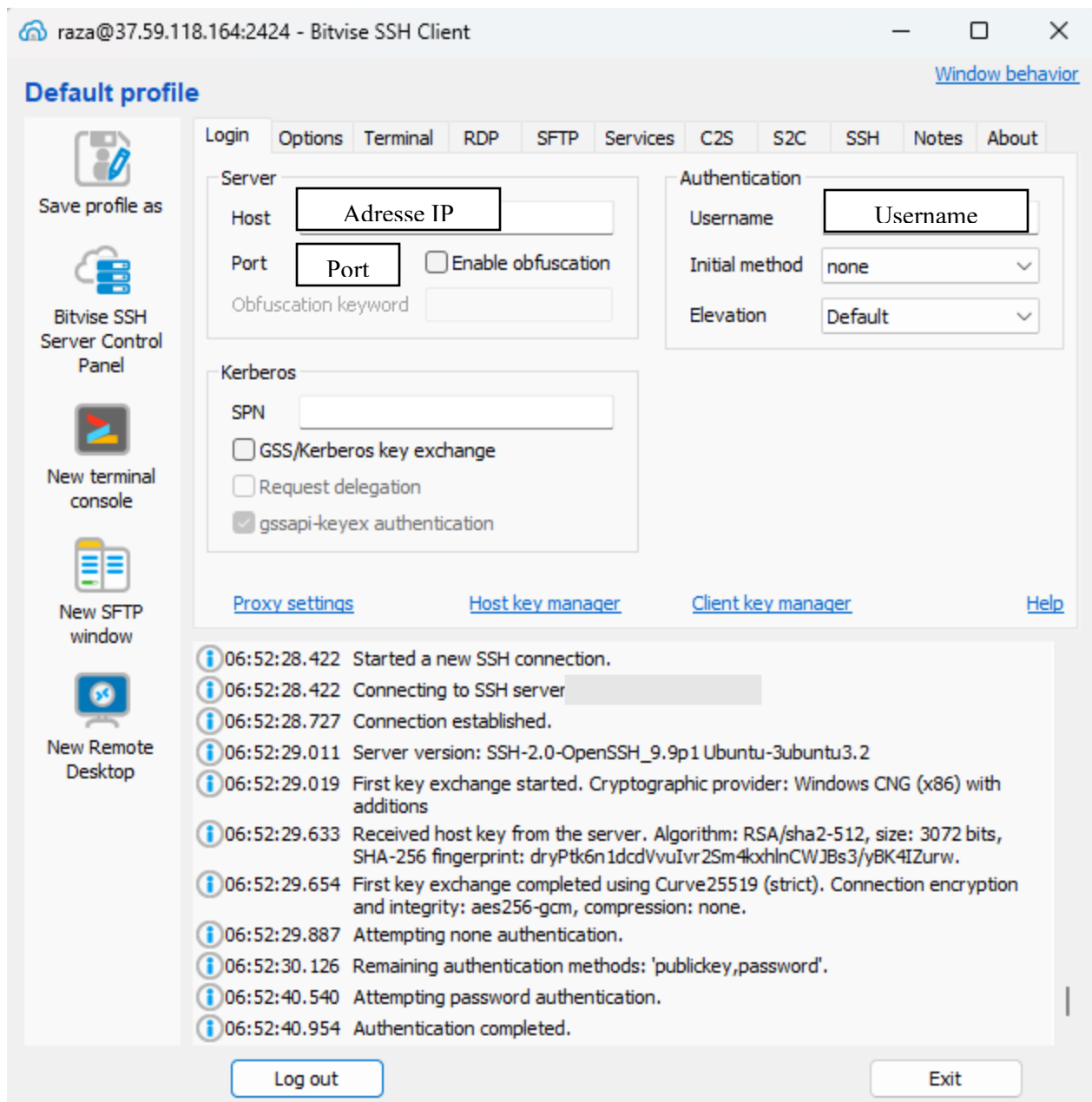
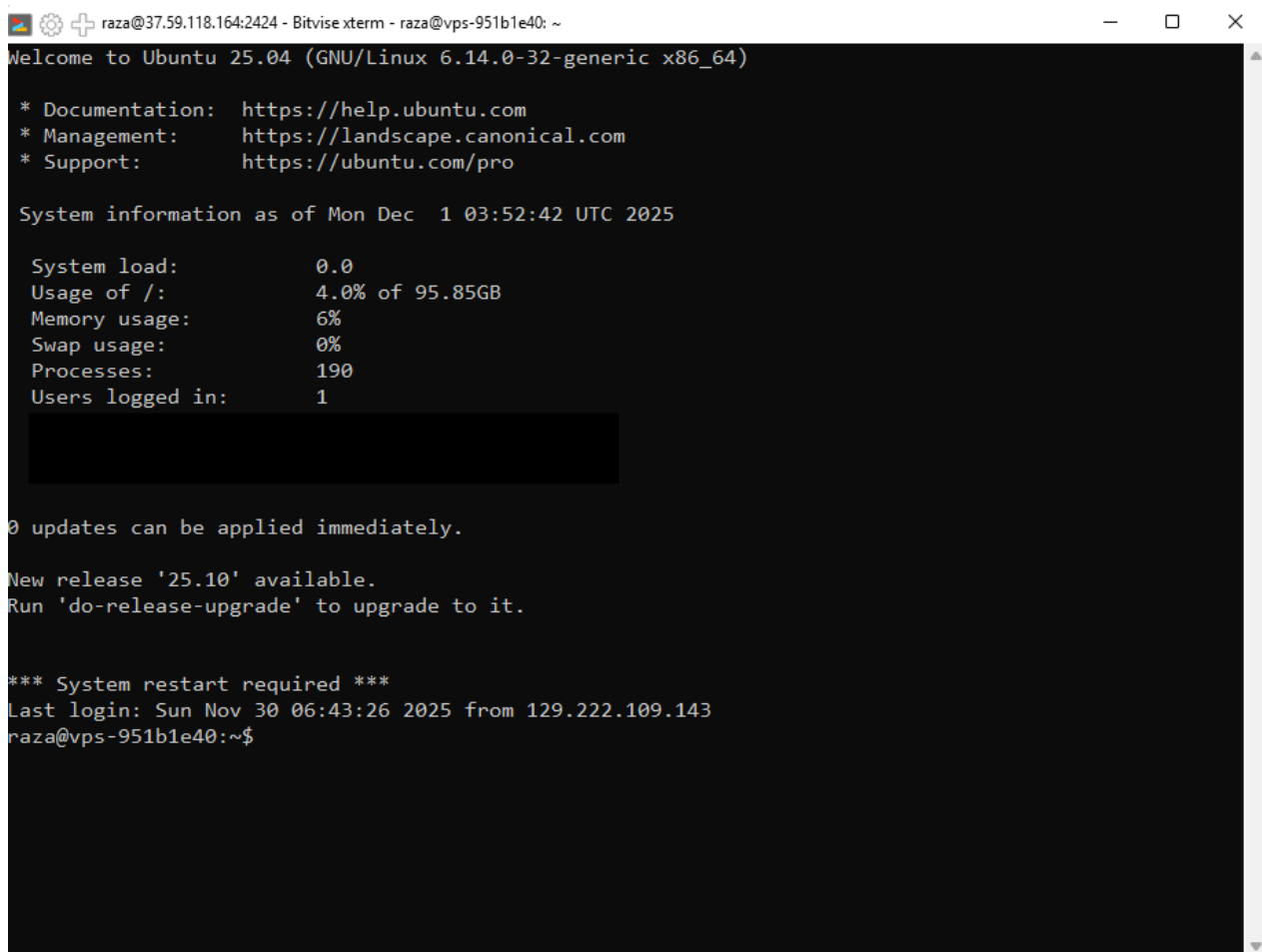


Figure 10 : Connexion établie

La figure montre l'interface terminal à l'intérieur de Bitvise, permettant d'exécuter des commandes directement sur le serveur. Elle met en évidence la capacité à naviguer dans les répertoires, gérer les fichiers et vérifier l'état du serveur.



```
raza@37.59.118.164:2424 - Bitvise xterm - raza@vps-951b1e40: ~
Welcome to Ubuntu 25.04 (GNU/Linux 6.14.0-32-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Mon Dec  1 03:52:42 UTC 2025

System load:          0.0
Usage of /:           4.0% of 95.85GB
Memory usage:         6%
Swap usage:           0%
Processes:            190
Users logged in:      1

0 updates can be applied immediately.

New release '25.10' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Sun Nov 30 06:43:26 2025 from 129.222.109.143
raza@vps-951b1e40:~$
```

Figure 11 : Terminal sur le serveur

L'utilisation de Bitvise SSH Client assure une connexion sécurisée et fiable au VPS OVHCloud. Les captures d'écran présentées illustrent clairement le processus depuis la configuration initiale jusqu'à l'accès au terminal du serveur. Cette étape est cruciale pour l'installation des outils, la configuration des services et le déploiement des conteneurs Docker, garantissant ainsi la sécurité et la stabilité de l'environnement de travail.

7.1.2. Git et configuration de GitHub

Dans le cadre de ce projet, **Git** a été retenu comme système de gestion de versions, permettant de suivre les modifications du code et de faciliter la collaboration entre les développeurs. Il était donc essentiel de choisir une plateforme d'hébergement de code favorisant la collaboration, l'intégration continue et la fiabilité. Après évaluation de plusieurs alternatives, le choix s'est porté sur **GitHub** comme solution d'hébergement.

GitHub se distingue par son interface conviviale et intuitive, qui simplifie le travail collaboratif et améliore la productivité des équipes. Son adoption massive dans l'industrie en fait une plateforme reconnue, largement documentée et soutenue, facilitant ainsi la prise en main et la résolution rapide d'éventuels problèmes.

L'utilisation du protocole **SSH** pour l'authentification constitue un atout majeur, garantissant une connexion sécurisée lors des opérations sur le dépôt. Cette approche protège les échanges et renforce la confidentialité des données du projet.

L'installation de Git est simple et rapide : il suffit de consulter le site officiel et de suivre les instructions adaptées au système d'exploitation utilisé. Cette procédure permet une mise en place efficace et sans difficulté dans l'environnement local.

En réunissant ces avantages – simplicité, sécurité et intégration fluide avec les outils existants – GitHub représente la solution la plus appropriée pour ce projet.

La figure suivante illustre l'installation de Git sur un système Windows 11, utilisé comme distribution locale dans notre environnement de développement.

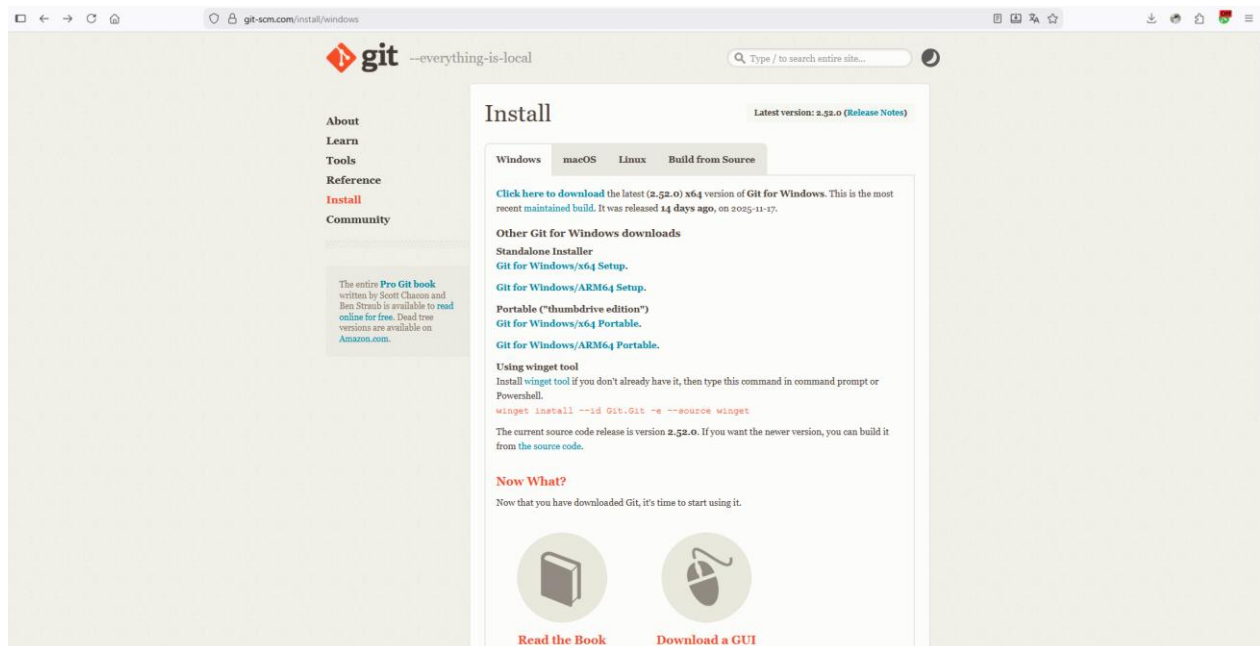


Figure 12 : Téléchargement et installation Git

L'utilisation d'un système de gestion de version fiable et sécurisé constitue une étape fondamentale dans tout projet de développement collaboratif. Dans ce cadre, la configuration

correcte de **Git** et l'établissement d'une connexion sécurisée via **SSH** avec **GitHub** sont essentiels pour garantir l'intégrité, la traçabilité et la confidentialité des contributions tout au long du projet.

Une fois l'installation de Git terminée, l'ouverture de l'application **Git Bash** permet de vérifier la réussite de l'opération grâce à la commande :

```
> git --version
```

L'affichage du numéro de version confirme que Git est correctement installé et fonctionnel.

L'association des commits à leur auteur est cruciale pour la traçabilité et la gestion des modifications. Cette identification s'effectue via les commandes :

```
> git config --global user.name "lolito"
```

```
> git config --global user.email "mamy.lolito@gmail.com"
```

Cette étape garantit que toutes les contributions seront correctement attribuées lors du travail collaboratif.

Pour sécuriser la communication avec GitHub, il est recommandé d'utiliser le protocole **SSH**. La clé est générée avec la commande :

```
> ssh-keygen -t ed25519 -C "mamy.lolito@gmail.com"
```

L'emplacement proposé par défaut peut être validé, et il est possible de laisser la passphrase vide ou d'en définir une pour renforcer la sécurité.

Pour que Git puisse utiliser la clé générée, l'agent SSH doit être lancé puis informé de la présence de la clé :

```
> eval $(ssh-agent -s)
```

```
> ssh-add ~/.ssh/id_ed25519
```

L'exécution de la commande suivante affiche la clé publique :

```
> cat ~/.ssh/id_ed25519.pub
```

Il convient ensuite de copier ce contenu et de l'ajouter à GitHub via **Settings > SSH and GPG Keys > New SSH key**. Ainsi, votre environnement local est explicitement autorisé à interagir avec votre espace GitHub.

La commande :

```
> ssh -T git@github.com
```

Permet de vérifier l'authentification. Un message de confirmation s'affiche si la configuration est correcte, attestant de la réussite de la liaison sécurisée.

En suivant scrupuleusement ces étapes, on assure la sécurisation des échanges, la traçabilité des contributions et une compatibilité totale entre l'environnement local de développement et la plateforme GitHub. Cette configuration constitue un gage de qualité et d'efficacité pour la gestion du code source dans le cadre de projets collaboratifs.

7.1.3. Visual Studio Code (VS Code)

Visual Studio Code (VS Code) est un éditeur de code open source multiplateforme, compatible avec Windows, macOS et Linux, qui s'est imposé comme l'outil de développement principal tout au long de ce projet. Pour une utilisation sous Windows, l'installation de VS Code s'effectue en téléchargeant l'installateur depuis le site officiel de Microsoft. Cette interface propose différentes versions adaptées aux besoins spécifiques de chaque système d'exploitation.

Après avoir téléchargé l'exécutable correspondant à Windows, il suffit de lancer l'installation et de suivre les instructions affichées à l'écran. Cette procédure permet d'obtenir un environnement moderne et personnalisable, intégrant des fonctionnalités avancées telles que l'assistance au codage, la gestion de projets, l'intégration avec Git et le support d'extensions variées.

VS Code offre ainsi une expérience optimisée pour le développement et les opérations sur Windows, tout en assurant une continuité dans la gestion du projet, la traçabilité du code et la collaboration avec l'équipe de développement.

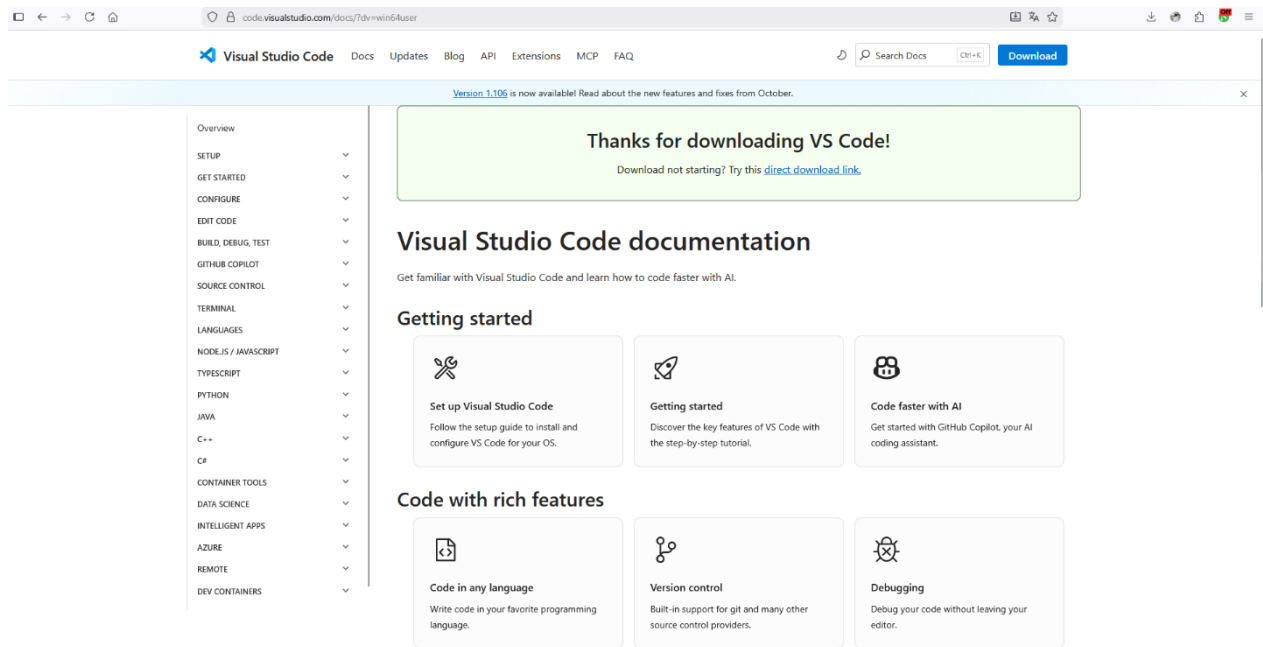


Figure 13 : Téléchargement et installation VSCode

7.1.4. Docker Desktop

Docker Desktop est une application permettant de créer, exécuter et gérer des conteneurs sur les systèmes Windows et macOS. Dans le cadre de ce projet, Docker Desktop a été choisi pour faciliter la conteneurisation des applications, simplifier le déploiement et assurer la cohérence sur l'environnement local.

a) Installation de Docker Desktop

- Téléchargement de l'installateur depuis le site officiel de Docker [<https://www.docker.com/products/docker-desktop>][6].
- Lancement de l'installation et suivi des instructions à l'écran pour compléter la configuration initiale.

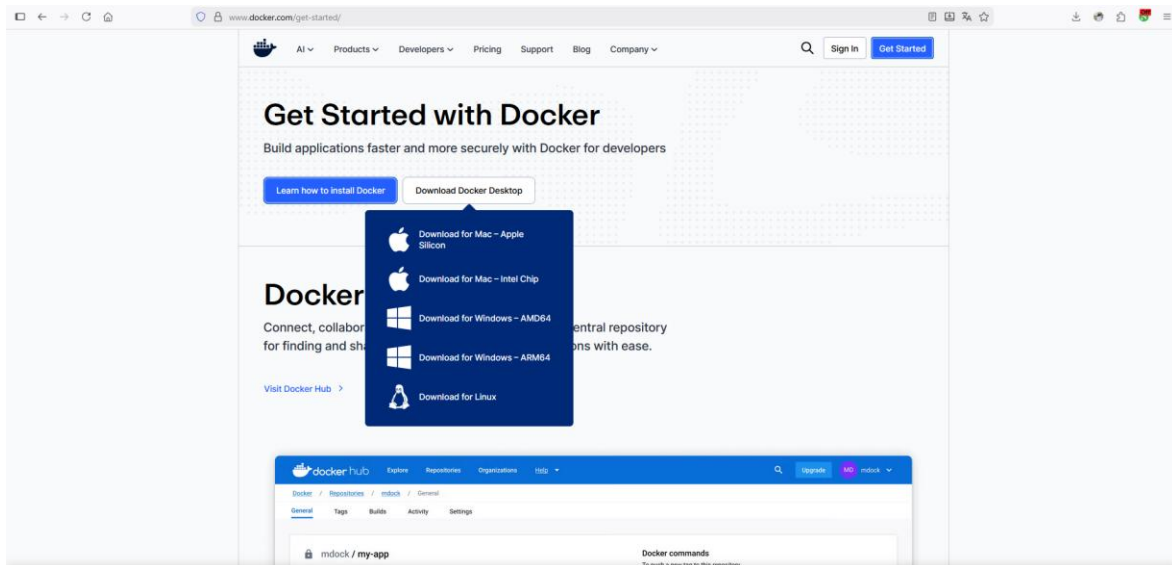


Figure 14 : Téléchargement et installation docker desktop

- Activation des fonctionnalités nécessaires telles que **WSL 2** (Windows Subsystem for Linux) pour une compatibilité complète avec les conteneurs Linux.

b) Configuration et utilisation

- Connexion à Docker Hub pour accéder aux images officielles et à celles créées pour le projet.

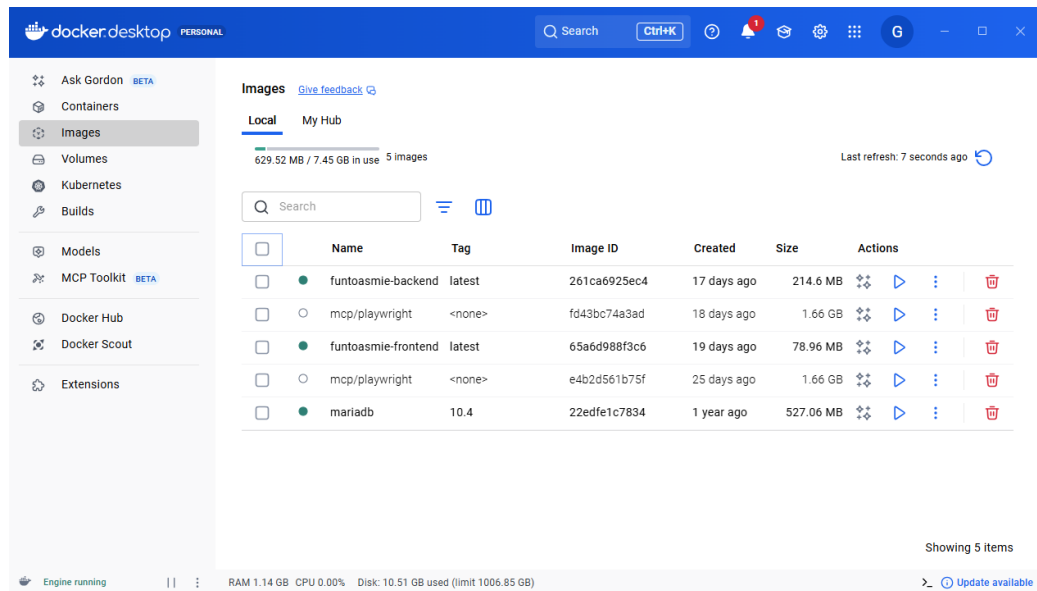


Figure 15 : Docker desktop image local

- Vérification du bon fonctionnement de Docker et Docker Compose à l'aide des commandes :

```
PS C:\Users\ghost> docker --version
Docker version 28.5.2, build ecc6942
PS C:\Users\ghost> docker-compose --version
Docker Compose version v2.40.3-desktop.1
PS C:\Users\ghost> |
```

- Création et exécution de conteneurs de test pour s'assurer que l'environnement est opérationnel.

Docker Desktop fournit une interface intuitive pour gérer les conteneurs, permet de standardiser l'environnement de développement et garantit la portabilité des applications sur la machine locale. Cette étape est cruciale pour la préparation du pipeline CI/CD et pour le déploiement efficace des services de l'application.

7.2. Conteneurisation avec Docker

La conteneurisation constitue une étape stratégique dans la modernisation du Système de Gestion des Services Médicaux Interentreprises. Elle permet de créer des environnements isolés, reproductibles et portables pour chaque composant de l'application, garantissant ainsi une cohérence totale entre les environnements de développement, de test et de production.

7.2.1. Installation et configuration de Docker sur le VPS

Après la connexion sécurisée au serveur VPS, Docker a été installé pour permettre la création et la gestion des conteneurs. Les étapes suivantes ont été suivies :

- **Mise à jour et sécurisation du système** : installation des dernières mises à jour et correctifs pour assurer la stabilité et la sécurité du serveur.

```
> sudo apt update && sudo apt upgrade -y
```

- **Installation des dépendances nécessaires** pour Docker : gestion des paquets HTTPS, certificats et outils essentiels.

```
> sudo apt install apt-transport-https ca-certificates curl software-properties-common -y
```

- **Ajout du dépôt officiel Docker et import de la clé GPG** pour garantir l'intégrité des paquets.

```
> curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg  
  
sudo add-apt-repository "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable"
```

- **Installation de Docker Engine et Docker Compose** pour disposer de l'ensemble des outils nécessaires à la conteneurisation et à l'orchestration multi-conteneurs.

```
> sudo apt update  
> sudo apt install docker-ce docker-ce-cli containerd.io docker-compose-plugin -y
```

- **Vérification de l'installation** via les commandes `docker --version` et `docker compose version` pour s'assurer que l'environnement est fonctionnel.

```
> docker --version  
> docker compose version
```

Cette étape assure que le serveur est prêt à exécuter des conteneurs de manière sécurisée et efficace, en respectant les standards professionnels pour un environnement de production.

7.2.2. Configuration des Dockerfile pour le Backend et le Frontend

Après l'installation de Docker sur le VPS, l'étape suivante consiste à préparer les environnements de construction des images nécessaires au déploiement de l'application. Pour cela, deux fichiers Dockerfile ont été créés : l'un pour le **backend Node.js**, l'autre pour le **frontend React.js**.

L'objectif est de fournir des images optimisées, reproductibles et adaptées à un environnement de production.

a) Dockerfile du Backend

Le Dockerfile backend permet de construire une image légère et sécurisée grâce à l'utilisation de l'image alpine. Il inclut uniquement les dépendances nécessaires à l'exécution en production.

```
backend > Dockerfile > ...
1  FROM node:20-alpine
2
3  LABEL version="1.0.0"
4  LABEL description="Backend API pour le système de gestion FUNTOA SMIE"
5
6  WORKDIR /app
7
8  COPY package*.json ./
9
10 RUN npm ci --omit=dev && npm cache clean --force
11
12
13 COPY . .
14
15 ENV NODE_ENV=production
16 EXPOSE 5000
17 CMD ["node", "index.js"]
```

Figure 16 : Dockerfile du backend

b) Dockerfile du Frontend

Le frontend React utilise un **build en deux phases** (multi-stage build) pour produire des fichiers statiques optimisés, ensuite servis par un serveur Nginx minimal.

```

frontend > Dockerfile > ...
1 FROM node:20-alpine AS builder
2 LABEL stage="build"
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci && npm cache clean --force
6 ARG REACT_APP_API_URL=http://localhost:5000/api
7 ENV REACT_APP_API_URL=${REACT_APP_API_URL}
8 ENV NODE_ENV=production
9 ENV CI=false
10 COPY . .
11 RUN printf "REACT_APP_API_URL=%s\n" "${REACT_APP_API_URL}" > .env.production && \
12     cat .env.production
13 RUN npm run build
14 RUN if [ ! -d "/app/build" ]; then \
15     echo "❌ ERREUR: Répertoire /app/build non trouvé après le build!"; \
16     ls -la /app/; \
17     exit 1; \
18 fi && \
19     echo "✅ Build réussi: /app/build existe" && \
20     ls -la /app/build | head -20
21 FROM nginx:1.27-alpine AS production
22 LABEL version="1.0.0"
23 LABEL description="Frontend FUNTOA SMIE"
24 LABEL maintainer="ilo-CS"
25 RUN rm -f /etc/nginx/conf.d/default.conf
26 COPY nginx/default.conf /etc/nginx/conf.d/default.conf
27 COPY --from=builder /app/build /usr/share/nginx/html
28 RUN if [ ! -f "/usr/share/nginx/html/index.html" ]; then \
29     echo "❌ ERREUR: /usr/share/nginx/html/index.html non trouvé!"; \
30     ls -la /usr/share/nginx/html/ || echo "Répertoire vide!"; \
31     exit 1; \
32 fi && \
33     echo "✅ Frontend files copié correctement" && \
34     ls -la /usr/share/nginx/html/
35 EXPOSE 80
36 CMD ["nginx", "-g", "daemon off;"]
37
38

```

Figure 17 : Dockerfile du frontend

7.2.3. Mise en place du fichier docker-compose.prod.yml

Pour orchestrer l'ensemble des services (frontend, backend, base de données, reverse proxy, etc.), un fichier **docker-compose.prod.yml** a été conçu.

Ce fichier permet le déploiement automatisé, cohérent et reproductible de l'application dans l'environnement de production.

L'objectif du docker-compose de production est de :

- Définir les services applicatifs et les réseaux internes.
- Configurer les variables d'environnement nécessaires.

- Gérer les volumes persistants (ex. MySQL).
- Faciliter le déploiement automatisé via GitHub Actions et GHCR.

services:

mysql:

image: mariadb:10.4

container_name: funtoa-mysql

restart: unless-stopped

environment:

MYSQL_ROOT_PASSWORD: \${DB_ROOT_PASSWORD:- }

MYSQL_DATABASE: \${DB_NAME:- }

MYSQL_USER: \${DB_USER:- }

MYSQL_PASSWORD: \${DB_PASSWORD:- }

ports:

- "\${MYSQL_PORT:-3306}:3306"

volumes:

- mysql_data:/var/lib/mysql

- ./backend/init.sql:/docker-entrypoint-initdb.d/init.sql:ro

networks:

- funtoa-network

healthcheck:

test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p\${DB_ROOT_PASSWORD:- }"]

```
interval: 10s

timeout: 5s

retries: 5


backend:

  image: ${BACKEND_IMAGE:-ghcr.io/ilo-cs/funtoa-backend:latest}

  container_name: funtoa-backend

  restart: unless-stopped

  depends_on:

    mysql:

      condition: service_healthy

  environment:

    NODE_ENV: development

    PORT: 5000

    DB_HOST: ${DB_HOST:-mysql}

    DB_PORT: ${DB_PORT:-3306}

    DB_USER: ${DB_USER:- }

    DB_PASSWORD: ${DB_PASSWORD:- }

    DB_NAME: ${DB_NAME:- }

    JWT_SECRET:${JWT_SECRET:-
952484620a2e86d77f88a86030bf96ebf91a4ae445e70a21e600a4cfba0bb1a1}

    JWT_EXPIRES_IN: ${JWT_EXPIRES_IN:-24h}

    JWT_REFRESH_EXPIRES_IN: ${JWT_REFRESH_EXPIRES_IN:-7d}
```

```

  CORS_ORIGIN: ${CORS_ORIGIN:-http://localhost,http://localhost:3000}

  BCRYPT_ROUNDS: ${BCRYPT_ROUNDS:-12}

  ports:

    - "${BACKEND_PORT:-5000}:5000"

  networks:

    - funtoa-network

  healthcheck:

    test: ["CMD", "node", "-e", "require('http').get('http://localhost:5000/health', (r) =>
{process.exit(r.statusCode === 200 ? 0 : 1)})"]

    interval: 30s

    timeout: 10s

    retries: 3

    start_period: 60s

  frontend:

    image: ${FRONTEND_IMAGE:-ghcr.io/ilo-cs/funtoa-frontend:latest}

    container_name: funtoa-frontend

    restart: unless-stopped

    depends_on:

      backend:

        condition: service_healthy

    environment:

      NODE_ENV: production

```

ports:

- "80:80"
- "443:443"

networks:

- funtoa-network

healthcheck:

test: ["CMD", "wget", "--quiet", "--tries=1", "--spider", "http://localhost/health"]

interval: 30s

timeout: 10s

retries: 3

start_period: 10s

volumes:

mysql_data:

driver: local

networks:

funtoa-network:

driver: bridge

Grâce à cette architecture Docker + Docker Compose, le serveur VPS dispose désormais d'un environnement standardisé, sécurisé et propice au déploiement automatisé. Cette configuration assure :

- Une cohérence entre l’environnement de développement et de production,
- Des mises à jour rapides via CI/CD,
- Une isolation complète entre les services,
- Une facilité de maintenance et d’évolution.

7.3. Mise en place d’un registre privé avec GitHub Container Registry (GHCR)

Dans une démarche DevOps moderne, l’utilisation d’un registre privé pour l’hébergement des images Docker constitue un élément central du pipeline de déploiement. Afin d’assurer la sécurité, la traçabilité et la maîtrise complète des artefacts applicatifs, le projet s’appuie sur **GitHub Container Registry (GHCR)**, un registre performant, intégré nativement à GitHub et parfaitement adapté aux environnements de production.

GHCR offre une gestion avancée des images conteneurisées, avec un contrôle d’accès granulaire, un hébergement privé, une haute disponibilité et une intégration complète avec GitHub Actions. L’adoption de ce registre permet ainsi de garantir un flux CI/CD cohérent, sécurisé et conforme aux bonnes pratiques industrielles.

7.3.1. Mécanisme d’authentification au registre

Pour permettre la communication sécurisée entre les environnements (VPS, pipeline GitHub Actions) et GHCR, un **Personal Access Token (PAT)** doit être généré. Celui-ci comporte les permissions suivantes :

- **read:packages** : lecture des images,
- **write:packages** : publication et mise à jour des images,
- **delete:packages** (optionnel) : suppression d’images obsolètes.

Le PAT est ensuite stocké de manière sécurisée dans GitHub Actions via l’onglet **Secrets**, ou exporté temporairement sur le VPS pour permettre l’authentification.

Select scopes

Scopes define the access for personal tokens. [Read more about OAuth scopes.](#)

☒ repo	Full control of private repositories
☒ repo:status	Access commit status
☒ repo_deployment	Access deployment status
☒ public_repo	Access public repositories
☒ repo:invite	Access repository invitations
☒ security_events	Read and write security events
☐ workflow	Update GitHub Action workflows
☒ write:packages	Upload packages to GitHub Package Registry
☒ read:packages	Download packages from GitHub Package Registry
☒ delete:packages	Delete packages from GitHub Package Registry

7.3.2. Intégration des images GHCR dans l'environnement de production

Le déploiement sur le VPS repose sur l'utilisation directe des images publiées dans GHCR. Grâce à Docker Compose, les services applicatifs consomment ces images via leur URI sécurisée :

```
services:
```

```
  backend:
```

```
    image: ghcr.io/utilisateur/backend:latest
```

```
  frontend:
```

```
    image: ghcr.io/utilisateur/frontend:latest
```

Cette approche garantit :

- Une uniformité totale entre les environnements développement, intégration et production,
- Une réduction des risques d'incohérence ou de dérive de configuration,
- Une rapidité de déploiement grâce au téléchargement direct depuis GHCR,
- Un processus de livraison maîtrisé et vérifiable.

La mise en place de GHCR comme registre privé constitue une étape clé dans l'automatisation et la sécurisation du cycle de déploiement. En garantissant la maîtrise complète des images Docker et leur distribution contrôlée, GHCR renforce la cohérence, la fiabilité et la professionnalisation de l'architecture DevOps retenue pour le projet.

7.4. Mise en place du pipeline CI/CD avec GitHub Actions

Dans le cadre d'une approche DevOps, l'automatisation du cycle de développement et de déploiement est un élément fondamental pour garantir la qualité, la rapidité et la fiabilité des livraisons. Pour ce projet, l'outil retenu pour l'intégration continue (CI) et le déploiement continu (CD) est **GitHub Actions**, une solution native à GitHub permettant d'automatiser l'ensemble du workflow de build, test, conteneurisation et publication des images dans GHCR.

L'objectif principal de ce pipeline est d'assurer qu'à chaque mise à jour du code source, l'application soit automatiquement construite, testée, conteneurisée, puis publiée sous forme d'images Docker prêtes à être déployées sur le serveur VPS.

Cette automatisation élimine les interventions manuelles, réduit les risques d'erreurs et garantit une cohérence totale entre les différentes versions déployées.

Le pipeline CI/CD mis en place avec GitHub Actions repose sur une architecture logique et séquentielle permettant d'automatiser l'ensemble des opérations allant de la récupération du code source jusqu'à la publication des images conteneurisées dans GHCR. Cette structure garantit un processus fluide, reproductible et conforme aux bonnes pratiques DevOps. Le pipeline s'exécute automatiquement lors d'un événement spécifique (généralement un *push* sur la branche principale) ou via un déclenchement manuel (*workflow dispatch*).

La structure globale du pipeline peut être présentée selon les phases suivantes :

Initialisation du workflow et récupération du code

Cette première étape consiste à déclencher l'exécution du pipeline et à cloner le dépôt GitHub contenant le code source.

Elle assure que l'environnement de travail dispose de la version la plus récente du projet, ce qui conditionne la validité de toutes les étapes ultérieures.

- **Préparation de l'environnement d'exécution**

GitHub Actions met en place un environnement Ubuntu standardisé. Cette étape fournit un contexte d'exécution contrôlé, garantissant la stabilité et la reproductibilité des opérations de build.

- **Authentification au registre GHCR**

Pour permettre la création et la publication des images Docker, l'environnement doit s'authentifier de manière sécurisée auprès du registre privé GHCR. L'authentification repose sur des secrets chiffrés, intégrés dans GitHub Actions, garantissant ainsi la confidentialité et la sécurité des accès au registre.

- **Construction des images Docker**

À partir des Dockerfile du frontend et du backend, le pipeline procède à la construction des images conteneurisées. Cette étape valide la qualité de la configuration des conteneurs et vérifie que le code peut être packagé correctement en images destinées à être déployées.

- **Publication des images vers GHCR**

Une fois construites, les images sont automatiquement envoyées vers GHCR. Cette phase assure la mise à disposition d'artefacts versionnés, centralisés et accessibles pour le déploiement en environnement de production.

- **Déclenchement du déploiement**

Dans un pipeline CD complet, une étape supplémentaire permettrait de déployer les nouvelles images directement sur le serveur VPS via Docker Compose. Cette phase garantit une livraison rapide et standardisée, sans intervention manuelle.

Les figures ci-dessous va nous montrer l'intégration et déploiement de notre application :

```

.github > workflows > deploy.yml > {} jobs > {} test-backend
19 #
20
21 on:
22   push:
23     branches:
24       - main
25       - master
26   workflow_dispatch:
27
28 env:
29   REGISTRY: ghcr.io
30   BACKEND_IMAGE: funtoa-backend
31   FRONTEND_IMAGE: funtoa-frontend
32   REPOSITORY_OWNER: ${ github.repository_owner }
33
34 jobs:
35   # =====
36   # CI STAGE 1: CODE VALIDATION
37   # =====
38   validate:
39     name: Validate Code
40     runs-on: ubuntu-latest
41     steps:
42       - name: Checkout code
43         uses: actions/checkout@v4
44         with:
45           fetch-depth: 0
46
47       - name: Validate Dockerfiles exist
48         run: |
49           if [ ! -f "./backend/Dockerfile" ]; then
50             echo "❌ Backend Dockerfile not found!"
51             exit 1
52           fi
53           if [ ! -f "./frontend/Dockerfile" ]; then
54             echo "❌ Frontend Dockerfile not found!"
55             exit 1
56           fi
57           echo "✅ All Dockerfiles found"
58
59       - name: Validate docker-compose files
60         run: |
61           if [ ! -f "./docker-compose.prod.yml" ]; then
62             echo "⚠️ docker-compose.prod.yml not found, but will use docker-compose.yml if available"
63           else
64             echo "✅ docker-compose.prod.yml found"
65           fi

```

Figure 18 : Job validate code

```

.github > workflows > deploy.yml > {} jobs > {} build-and-push > {} steps > {} 4 > abc uses
34 jobs:
164 build-and-push:
165   name: Build and Push Docker Images
166   needs: validate
167   runs-on: ubuntu-latest
168   permissions:
169     contents: read
170     packages: write
171
172   steps:
173     - name: Checkout code
174       uses: actions/checkout@v4
175       with:
176         fetch-depth: 0
177
178     - name: Set up Docker Buildx
179       uses: docker/setup-buildx-action@v3
180
181     - name: Log in to GitHub Container Registry
182       uses: docker/login-action@v3
183       with:
184         registry: ${ env.REGISTRY }
185         username: ${ github.actor }
186         password: ${ secrets.GITHUB_TOKEN }
187
188     - name: Set repository owner (lowercase)
189       id: repo-owner
190       run: |
191         REPO_OWNER_LOWER=$(echo "${ github.repository_owner }" | tr '[:upper:]' '[:lower:]')
192         echo "owner=$REPO_OWNER_LOWER" >> $GITHUB_OUTPUT
193         echo "✅ Repository owner: ${ github.repository_owner } -> $REPO_OWNER_LOWER"
194
195     - name: Extract metadata for backend
196       id: meta-backend
197       uses: docker/metadata-action@v5
198       with:
199         images: ${ env.REGISTRY }/${ steps.repo-owner.outputs.owner }/${ env.BACKEND_IMAGE }
200         tags: |
201           type=ref,event=branch
202           type=ref,event=pr
203           type=semver,pattern={{version}}
204           type=semver,pattern={{major}}.{{minor}}
205           type=sha,prefix={{branch}}-
206           type=raw,value=latest,enable={{is_default_branch}}
207
208     - name: Build and push backend image
209       uses: docker/build-push-action@v5
210       with:
211         context: ./backend

```

Figure 19 : Job build and push docker image

```

.github > workflows > deploy.yml > {} jobs > {} build-and-push > {} steps > {} 6 > {} with > abc tags
34 jobs:
164   build-and-push:
244
245   deploy:
246     name: Deploy to VPS
247     needs: build-and-push
248     runs-on: ubuntu-latest
249     if: github.ref == 'refs/heads/main' || github.ref == 'refs/heads/master'
250     timeout-minutes: 15
251
252     steps:
253       - name: Checkout code
254         uses: actions/checkout@v4
255
256       # -----
257       # ● CD STAGE 2: VALIDATE DEPLOYMENT SECRETS
258       # -----
259       - name: Validate VPS and registry secrets
260         run: |
261           if [ -z "${{ secrets.IP_SRV }}" ]; then
262             echo "❌ Secret IP_SRV is not set!"
263             exit 1
264           fi
265           if [ -z "${{ secrets.USR_SRV }}" ]; then
266             echo "❌ Secret USR_SRV is not set!"
267             exit 1
268           fi
269           if [ -z "${{ secrets.SSH_PK }}" ]; then
270             echo "❌ Secret SSH_PK is not set!"
271             exit 1
272           fi
273           if [ -z "${{ secrets.PORT_SRV }}" ]; then
274             echo "❌ Secret PORT_SRV is not set!"
275             exit 1
276           fi
277           # GHCR_TOKEN should be set as a PAT with read:packages for the server to pull private images
278           if [ -z "${{ secrets.GHCR_TOKEN }}" ]; then
279             echo "❌ Secret GHCR_TOKEN is not set! Please provide a PAT with read:packages (GHCR_TOKEN)."
280             exit 1
281           fi
282           echo "✅ All required secrets are set"
283           echo "✅ SSH Port configured: ${{ secrets.PORT_SRV }}"
284
285       # -----
286       # ● CD STAGE 3: SSH DEPLOY SCRIPT
287       # -----
288       # Execute remote deployment script on VPS via SSH
289       - name: Deploy to VPS

```

Figure 20 : Job deploy to VPS

Chapitre 8: Test et validation

Ce chapitre présente les différentes phases de tests et de validation réalisées tout au long du projet FUNTOA SMIE. L'accent sera mis sur la mise en place des processus d'intégration continue et de déploiement continu (CI/CD) pour le backend et le frontend, ainsi que sur les tests d'accessibilité et de performance de l'application.

8.1. Test sur l'intégration continue et le déploiement continu (CI/CD)

Le pipeline d'intégration et de déploiement continus repose sur GitHub Actions, assurant ainsi une intégration directe avec GitHub pour l'automatisation des workflows de déploiement. Ce pipeline s'active lors d'événements tels que les actions de type « push ».

La figure 21 illustre l'exécution du pipeline consécutive à une action de « push ».

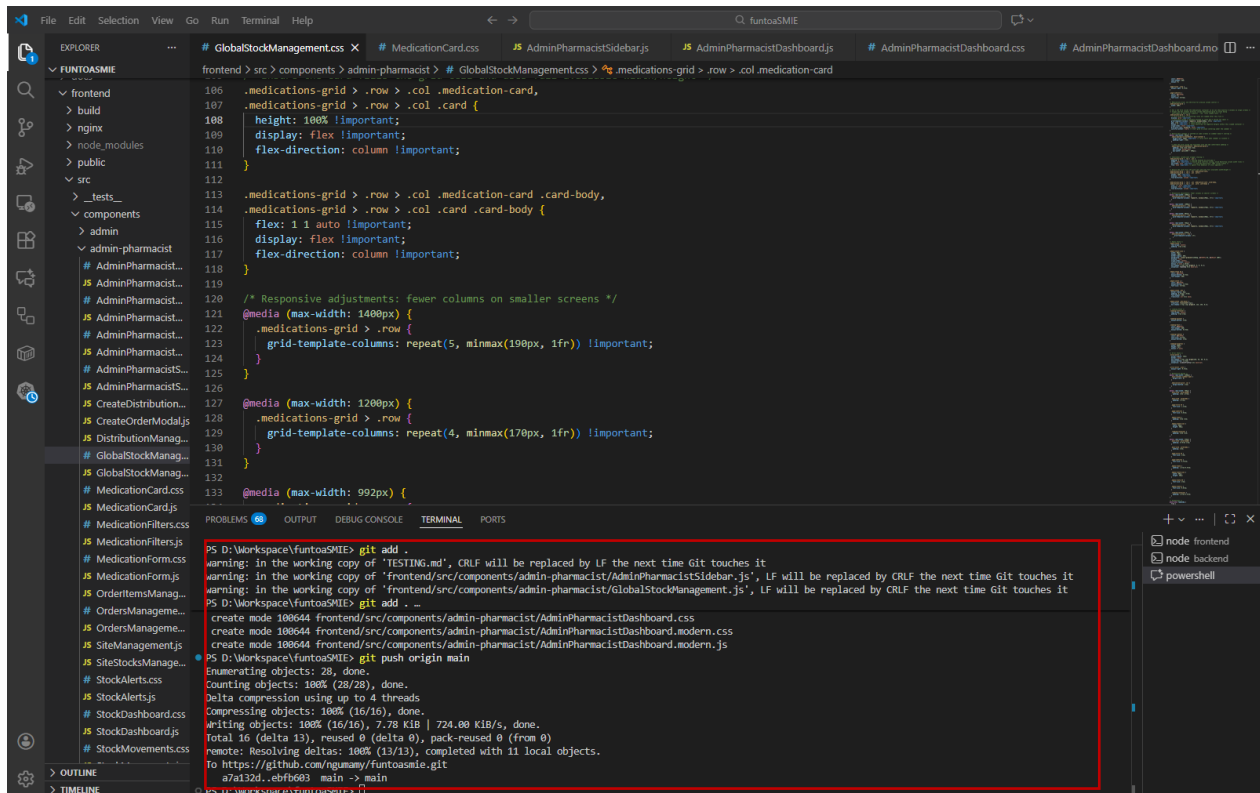


Figure 21 : Action Push vers Central Hub (Github)

Dès qu'un push est effectué sur la branche principale, le pipeline se déclenche automatiquement pour lancer le processus de mise à jour de l'application. Ce pipeline gère successivement les différentes étapes, telles que la compilation, les tests et le déploiement, ce qui

permet d'intégrer les modifications de manière transparente et sans intervention manuelle. Les figures 22,23 et 24 illustrent les principales étapes du pipeline, mettant en lumière les phases de compilation et de déploiement, et démontrent ainsi l'efficacité de l'intégration continue et du déploiement continu (CI/CD) dans le cycle de vie de l'application. Ainsi, dès que le code est validé et fusionné sur la branche principale, le pipeline s'exécute automatiquement, assurant une synchronisation transparente entre les évolutions du code et la mise à jour de l'application.

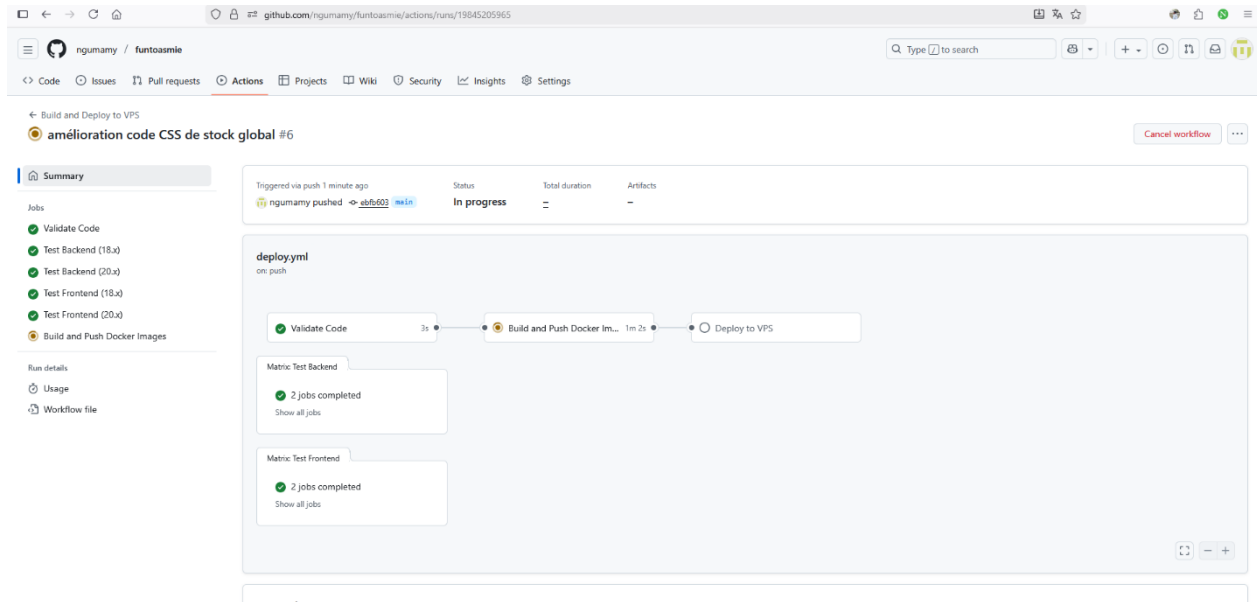


Figure 22 : Phase de build du pipeline en cours d'exécution

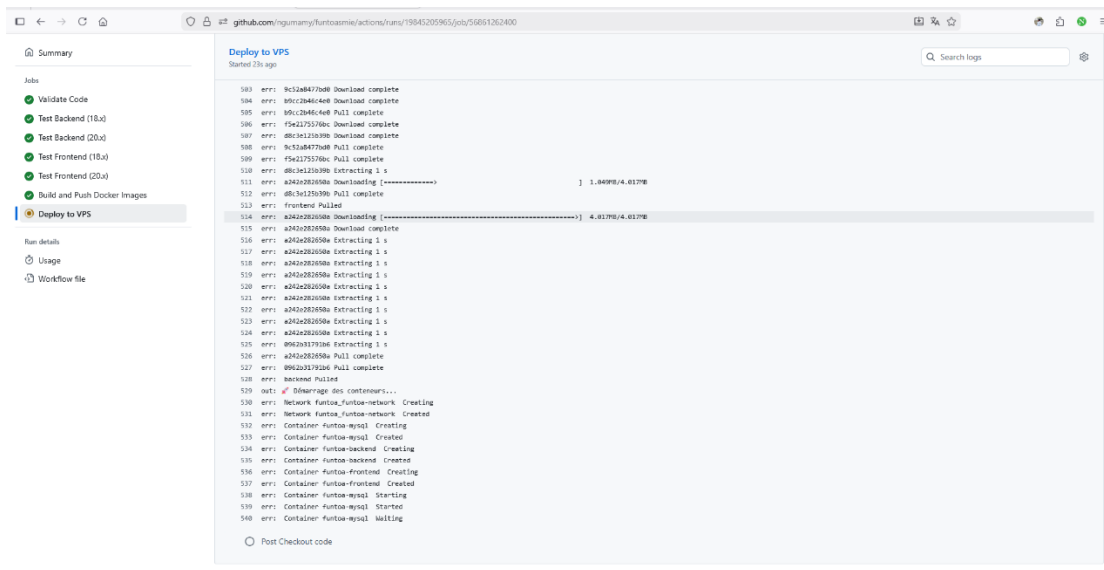


Figure 23 : Phase de build du pipeline en cours d'exécution

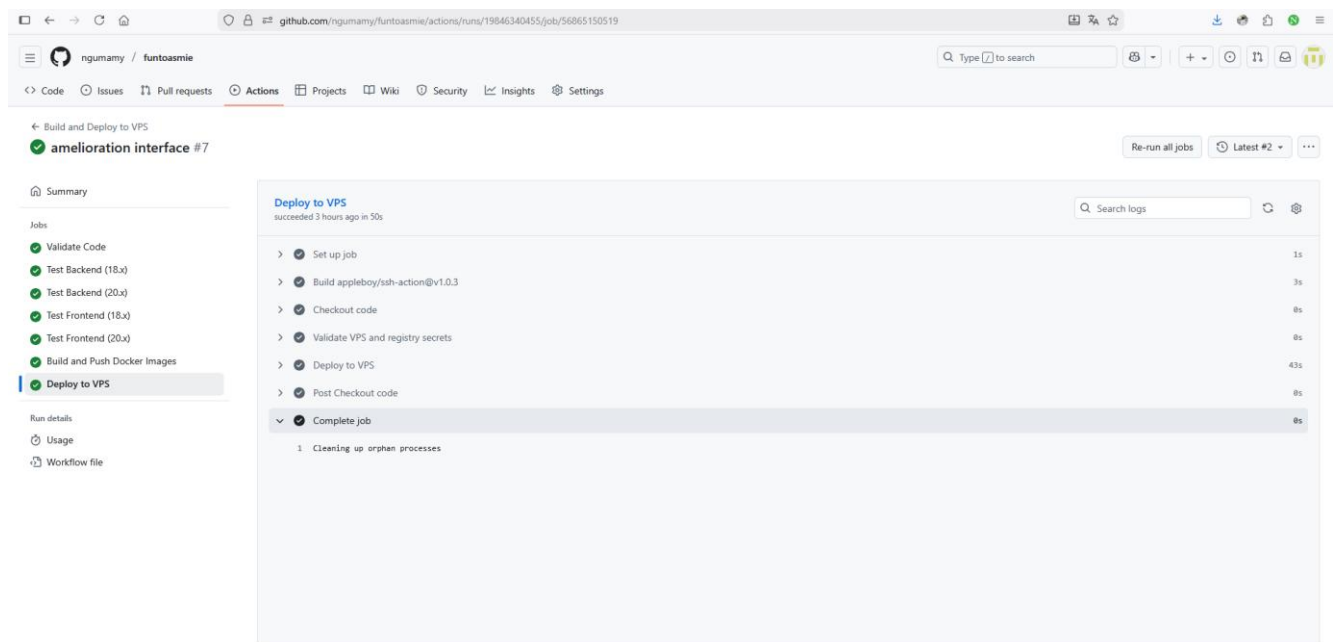


Figure 24 : Succès du pipeline

8.2. Tests et validation d'accessibilité de l'application

Après l'exécution réussie du pipeline CI/CD, l'application est automatiquement déployée sur le serveur VPS, garantissant ainsi une continuité entre les environnements de développement, de test et de production. L'intégration systématique de ces processus au cycle de vie du développement permet non seulement d'accélérer la mise en production des nouvelles fonctionnalités, mais aussi de maintenir un niveau de contrôle qualité rigoureux à chaque étape, grâce à la vérification automatique des tests unitaires, fonctionnels et d'intégration. Cette approche assure la stabilité, la fiabilité et la cohérence du système tout au long de son évolution. L'accès à l'application pour les utilisateurs autorisés est disponible à l'adresse suivante : <http://app.funtoa-smie.com/>, offrant une interface fonctionnelle conforme aux exigences du projet FUNTOA SMIE.

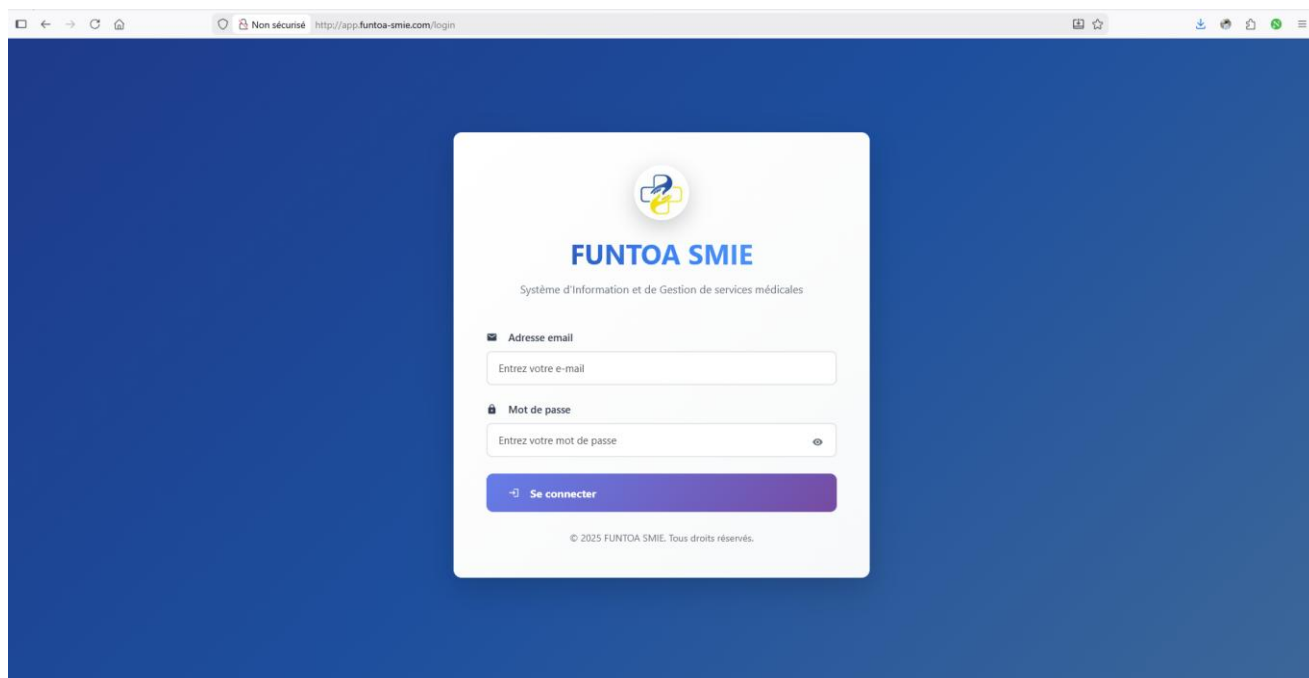


Figure 25 : Login

8.3. Test d'amélioration du code et validation comparative

Dans le cadre de l'amélioration continue du projet FUNTOA SMIE, un **test d'amélioration du code** a été réalisé afin de garantir la qualité, la performance et l'accessibilité de l'application. Pour ce faire, le code source local a été capturé et intégré au pipeline CI/CD, permettant ainsi de **déclencher automatiquement les tests et le déploiement**.

8.3.1. État initial du code déployé

Avant toute modification, l'application déployée sur le nom de domaine a été capturée et analysée pour documenter son état initial, identifier d'éventuelles anomalies et servir de référence pour mesurer les améliorations ultérieures.

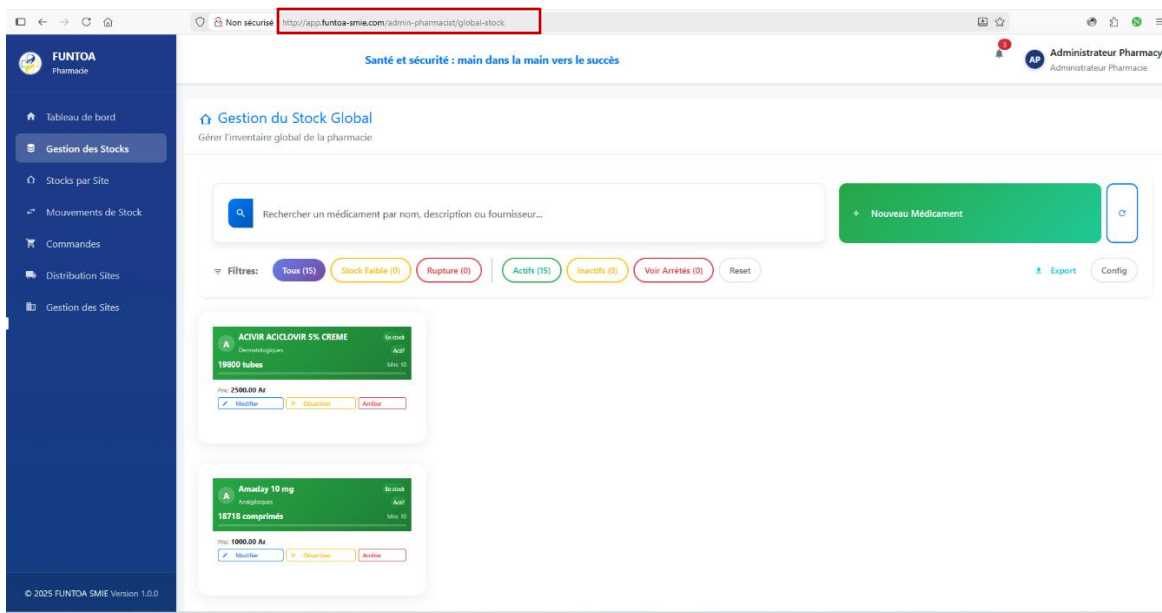


Figure 26 : État initial du code déployé

8.3.2. Améliorations et tests après modification

Après l'analyse de l'état initial, le code a été modifié et testé localement pour vérifier que les améliorations fonctionnent correctement, que l'accessibilité est optimisée et que le système reste stable avant le déploiement.

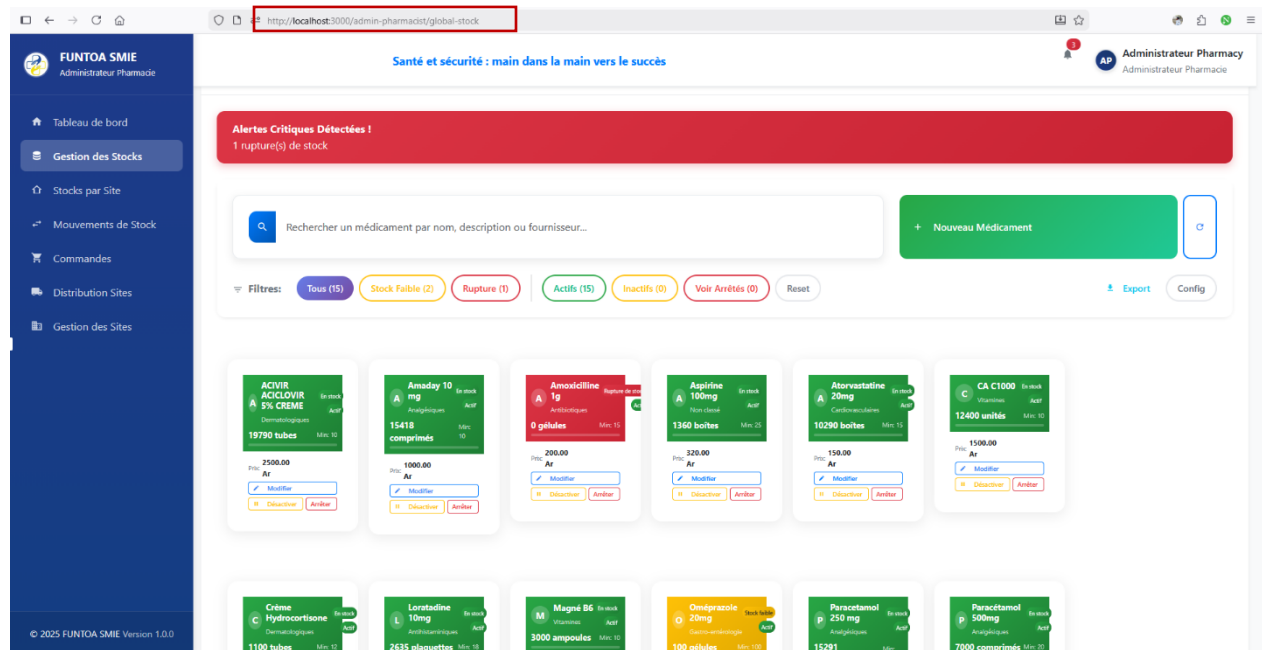


Figure 27 : Améliorations et tests après modification

8.3.3. Validation des améliorations mises en production sur le nom de domaine

Après validation locale, la version améliorée de l'application a été déployée sur le nom de domaine pour vérifier que les améliorations fonctionnent correctement, que l'accessibilité et la performance sont optimisées, et pour comparer l'état initial à l'état amélioré.

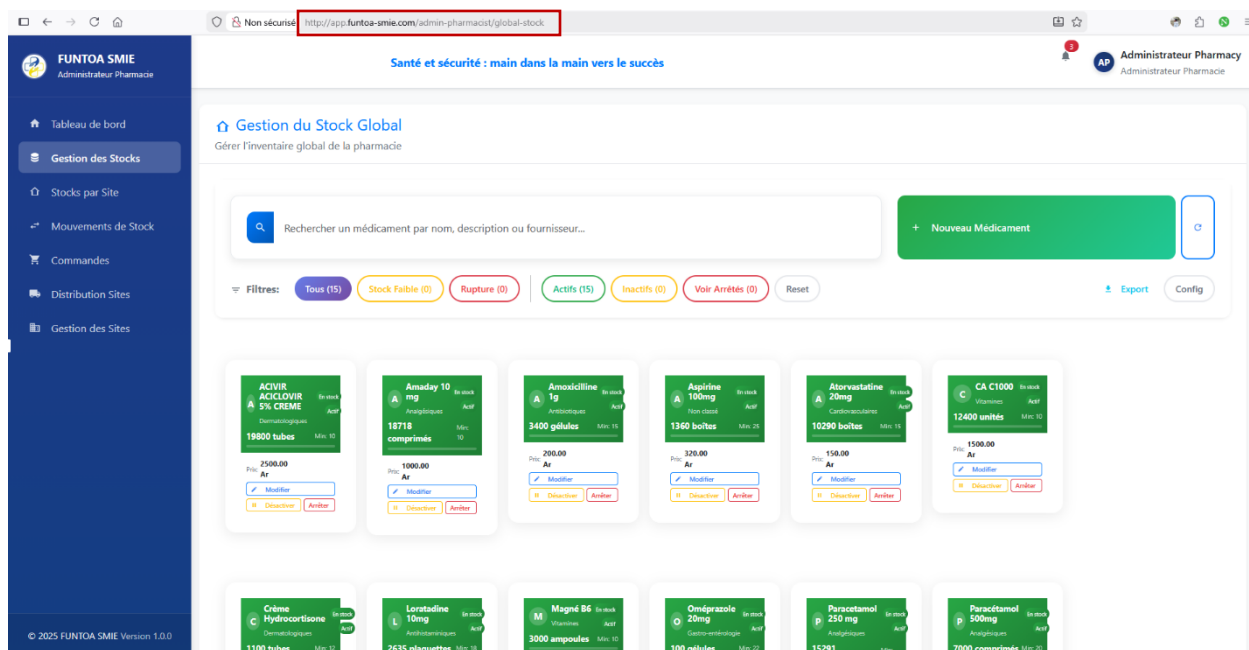


Figure 28 : Validation des améliorations mises en production sur le nom de domaine

CONCLUSION

En conclusion, ce mémoire a mis en évidence l'importance stratégique de la modernisation technologique des systèmes d'information au sein d'une organisation telle que le SMIE. La mise en place d'une chaîne CI/CD automatisée, articulée autour de GitHub Actions, Docker et d'un environnement VPS, a démontré sa capacité à offrir une solution de déploiement fiable, flexible et évolutive, parfaitement adaptée aux exigences actuelles en matière d'efficacité opérationnelle et de continuité de service.

Les travaux réalisés ont confirmé que l'intégration des pratiques DevOps représente un véritable catalyseur d'optimisation pour le cycle de vie applicatif. L'automatisation des processus, la réduction des interventions manuelles et la standardisation des environnements ont permis d'améliorer significativement la qualité logicielle, tout en minimisant les risques d'erreurs et les délais de mise en production. Les tests et validations menés ont attesté de la performance, de la stabilité et de la scalabilité de l'infrastructure développée, renforçant la pertinence des choix techniques et méthodologiques adoptés.

Au-delà des aspects techniques, ce projet illustre le rôle déterminant que jouent l'automatisation, la containerisation et les bonnes pratiques DevOps dans la transformation numérique des organisations. Ces approches apportent non seulement une réponse concrète aux défis actuels liés à la gestion des systèmes distribués, mais ouvrent également la voie à de futures évolutions : intégration d'outils avancés de monitoring, renforcement de la sécurité, amélioration continue des pipelines, ou encore mise en place d'une architecture plus résiliente et orientée microservices.

Ainsi, cette étude démontre que l'adoption cohérente et progressive des outils DevOps constitue un levier majeur pour accompagner la modernisation continue des systèmes d'information et soutenir durablement la performance opérationnelle du SMIE. Elle pose les fondations nécessaires à la poursuite d'une stratégie numérique ambitieuse, capable de s'adapter aux évolutions technologiques et aux besoins croissants des utilisateurs dans un environnement en perpétuelle mutation.

BIBLIOGRAPHIE

- [1] Gene Kim, Patrick Debois, John Willis, Jez Humble, « The Phoenix Project : A Novel About IT, DevOps, and Helping Your Business Win, » IT Revolution Press, 2013.
- [2] Kim, G., Humble, J., Debois, P., & Willis, J., «The DevOps Handbook: How to Create World-Class Agility, Reliability, & Security in Technology Organizations,» IT Revolution Press, p. 408, 2016.
- [3] Puppet, «State of DevOps Report 2018.,» Puppet., 2018.
- [4] A. Hajdin, Docker, 2024
- [5] Humble, J., & Farley, D., «Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation.,» Addison-Wesley, 2010.

WEBOGRAPHIE

- [6] <https://docs.docker.com/get-started/get-docker/>.
- [7] <https://medium.com/@unosega/ci-cd-deployment-for-mern-stack-application-using-docker-and-github-actions-on-a-remote-server-10a5a2b31d6f>
- [8] <https://github.com>
- [9] <https://code.visualstudio.com/>
- [10] <https://www.ovhcloud.com/fr/>
- [11] <https://aws.amazon.com/fr/>

GLOSSAIRE

CI/CD (Continuous Integration / Continuous Deployment) : Pratiques DevOps visant à automatiser l'intégration du code et le déploiement des applications pour améliorer la rapidité, la qualité et la fiabilité des livraisons logicielles.

Containerisation : Technique consistant à isoler les applications et leurs dépendances dans des conteneurs (ex : Docker) afin d'assurer portabilité, cohérence et scalabilité.

DevOps : Ensemble de pratiques combinant le développement logiciel (Dev) et les opérations IT (Ops), visant à améliorer la collaboration, l'automatisation et la qualité du cycle de vie des applications.

Docker : Plateforme de containerisation permettant de créer, déployer et exécuter des applications dans des conteneurs légers et portables.

GitHub Actions : Service d'automatisation intégré à GitHub permettant de mettre en place des workflows CI/CD pour le test, la construction et le déploiement d'applications.

Infrastructure as Code (IaC) : Pratique qui consiste à gérer et provisionner les ressources informatiques via du code, facilitant la reproductibilité et l'automatisation des environnements.

Orchestration : Gestion automatisée des conteneurs et des services, souvent via des outils comme Kubernetes, pour assurer la scalabilité, la résilience et la disponibilité des applications.

Serveur VPS (Virtual Private Server) : Serveur virtuel offrant des ressources dédiées sur un serveur physique partagé, utilisé pour héberger des applications et services de manière sécurisée et flexible.

Pipeline CI/CD : Ensemble d'étapes automatisées permettant de tester, construire et déployer le code applicatif de manière continue.

Scalabilité : Capacité d'un système à s'adapter à une augmentation de la charge ou du volume de travail sans dégradation de la performance.

Transformation numérique : Processus par lequel une organisation adopte des technologies numériques pour améliorer ses processus, ses services et sa compétitivité.

ANNEXE

Annexe 1 : Logo du FUNTOA Service Médical Interentreprises (SMIE)



Annexe 2 : Logo de l'Ecole Nationale D'informatique



Annexe 3 – Topologie Réseau du FUNTOA Service Médical Interentreprises (SMIE)

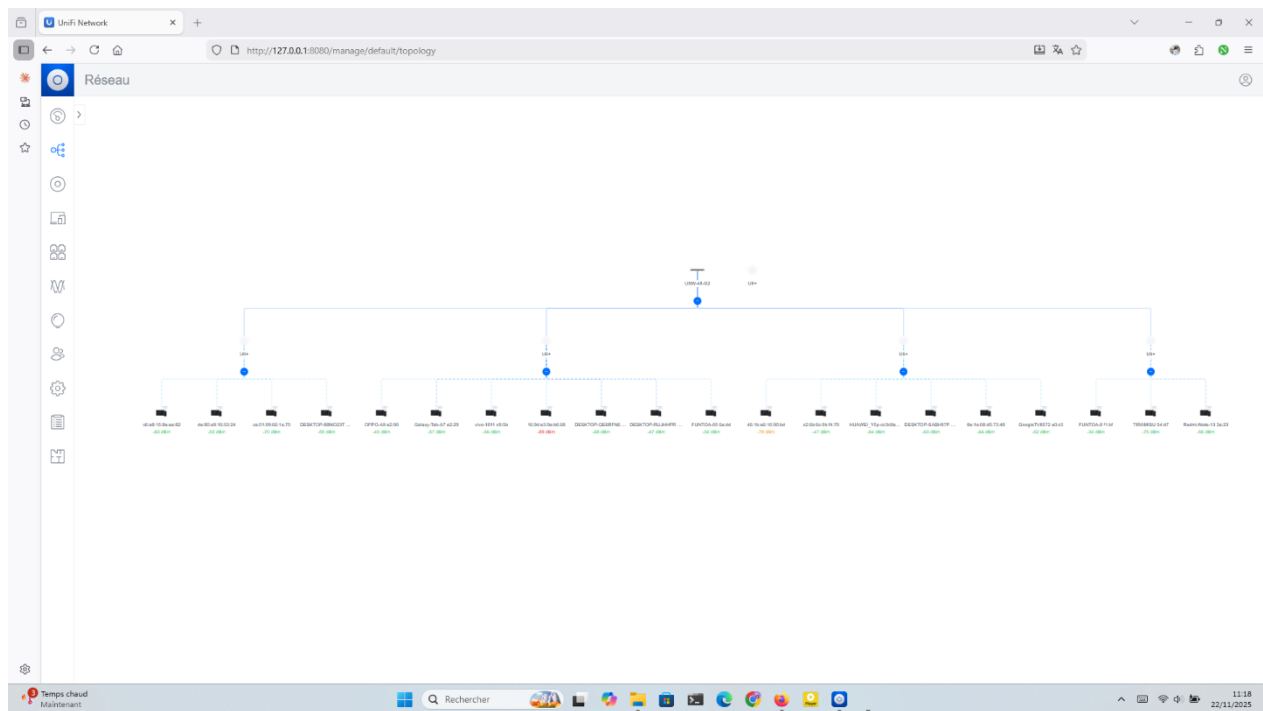


TABLE DES MATIERES

CIRRICULUM VITAE	i
SOMMAIRE	iv
REMERCIEMENTS	vi
LISTE DES FIGURES	vii
LISTE DES TABLEAUX	viii
LISTE DES ABREVIATIONS.....	ix
INTRODUCTION GENERALE.....	1
PARTIE I : PRESENTATIONS	2
Chapitre 1: Présentation de l'Ecole Nationale d'Informatique	3
1.1. Information d'ordre générale	3
1.2. Missions et historiques	3
1.3. Organigramme institutionnel.....	5
1.4. Domaine de spécialisation.....	6
1.5. Architecture des formations pédagogiques	7
1.6. Relation de l'ENI avec les organismes externes	8
1.7. Débouchés professionnels et diplômés	9
1.8. Ressources humaines.....	11
Chapitre 2: Présentation de SMIE FUNTOA.....	13
2.1. Statut légal et identification de l'organisation.....	13
2.2. Vision, mission et philosophie d'intervention	14
2.2.1. Mission principale	14
2.2.2. Philosophie et valeurs.....	15
2.3. Entreprises partenaires et réseau d'intervention nationale	15
2.4. Ressources humaines et organisation interne	16

2.5. Atouts stratégiques et avantages concurrentiels	16
2.5.1. Compétitivité financière	16
2.5.2. Infrastructures médicales modernes et diversifiées	17
2.5.3. Réactivité et excellence opérationnelle	17
2.5.4. Engagement en matière de qualité et d'éthique médicale	17
2.6. Offre globale des services médicaux	17
2.6.1. Couverture médicale complète	17
2.6.2. Services exclusifs	18
2.6.3. Médecine du travail et prévention	18
2.6.4. Gestion des doléances	18
2.7. Perspective de transformation digitale	18
Chapitre 3: Présentation et cadrage du projet.....	20
3.1. Contexte général du projet	20
3.2. Objectif et besoins de l'utilisateur.....	20
3.3. Moyens nécessaires à la réalisation du projet	21
3.3.1. Moyens techniques	21
3.3.2. Moyens humains	22
3.3.3. Moyens matériels	24
3.3.4. Moyens logiciels	25
3.4. Résultats attendus.....	26
3.5. Chronogramme des activités	27
PARTIE II : ANALYSE ET CHOIX DES OUTILS	30
Chapitre 4: Analyse préalable et conception fonctionnelle.....	31
4.1. Analyse de l'existant	31
4.1.1. Organisation actuelle.....	31

4.1.2. Inventaire des moyens matériels et logiciels	31
a) Postes de travail et périphériques	32
b) Logiciels utilisés	32
c) Réseau et connectivité	33
4.2. Critique de l'existant	33
4.3. Proposition des solutions.....	34
4.3.1. Centralisation et modernisation des données	34
4.3.2. Développement structuré de l'application.....	34
4.3.3. Mise en place d'une pipeline CI/CD	34
4.3.4. Automatisation et conteneurisation	35
4.4. Conception avant la réalisation de l'application	35
4.4.1. Méthode de modélisation (UML).....	35
4.4.2. Langage de programmation.....	36
4.4.3. Système de gestion de base de données	36
4.4.4. Outils de conception.....	37
4.5. Conception du système.....	38
4.5.1. Diagramme de cas d'utilisation global.....	38
Chapitre 5: Revue de littérature	40
5.1. Modernisation des systèmes d'information.....	40
5.2. Approche DevOps	41
5.2.1. Définition	41
5.2.2. Avantages de l'approche DevOps.....	41
5.2.3. Cycle de vie DevOps.....	42
Chapitre 6: Choix des outils	44
6.1. Serveur VPS – OVHCloud.....	44

6.1.1. Justification du choix du fournisseur OVHCloud	44
6.1.2. Performance et caractéristiques techniques du VPS	45
6.1.3. Sécurité et accès au serveur	45
6.1.4. Compatibilité avec Docker et la CI/CD	45
6.1.5. Scalabilité et maîtrise de l'infrastructure	45
6.2. Outils de conteneurisation	46
6.2.1. Docker	46
6.2.2. Docker Compose	47
6.3. Répertoire d'images Docker – GitHub Container Registry (GHCR)	48
6.3.1. Présentation du GHCR	48
6.3.2. Justification du choix	49
6.3.3. Comparaison avec d'autres registres Docker	49
6.4. Intégration Continue (CI) & Déploiement Continu (CD)	50
6.4.1. Intégration Continue (CI)	50
6.4.2. Déploiement Continu (CD)	51
6.4.3. Comparaison avec d'autres outils CI/CD	51
6.4.4. Justification du choix de GitHub Actions	52
6.5. Outils de développement	53
6.5.1. Backend – Node.js	53
6.5.2. Frontend – React.js	54
6.5.3. Base de données – MySQL	54
6.5.4. Reverse proxy – Nginx	55
PARTIE III : MISE EN ŒUVRE	57
Chapitre 7: Réalisation du projet	58
7.1. Préparation de l'environnement	58

7.1.1. Serveur VPS OVHCloud.....	58
a) Acquisition du serveur.....	58
b) Connexion au serveur VPS avec Bitwise SSH Client	60
7.1.2. Git et configuration de GitHub.....	63
7.1.3. Visual Studio Code (VS Code).....	66
7.1.4. Docker Desktop.....	67
a) Installation de Docker Desktop	67
b) Configuration et utilisation	68
7.2. Conteneurisation avec Docker	69
7.2.1. Installation et configuration de Docker sur le VPS.....	69
7.2.2. Configuration des Dockerfile pour le Backend et le Frontend.....	70
a) Dockerfile du Backend.....	71
b) Dockerfile du Frontend	71
7.2.3. Mise en place du fichier docker-compose.prod.yml	72
7.3. Mise en place d'un registre privé avec GitHub Container Registry (GHCR).....	77
7.3.1. Mécanisme d'authentification au registre	77
7.3.2. Intégration des images GHCR dans l'environnement de production.....	78
7.4. Mise en place du pipeline CI/CD avec GitHub Actions.....	79
Chapitre 8: Test et validation.....	84
8.1. Test sur l'intégration continue et le déploiement continu (CI/CD)	84
8.2. Tests et validation d'accessibilité de l'application	86
8.3. Test d'amélioration du code et validation comparative.....	87
8.3.1. État initial du code déployé.....	87
8.3.2. Améliorations et tests après modification	88
8.3.3. Validation des améliorations mises en production sur le nom de domaine	89

CONCLUSION	90
BIBLIOGRAPHIE	xi
WEBOGRAPHIE	xii
GLOSSAIRE	xiii
ANNEXE	xiv
TABLE DES MATIERES	xv
RESUME	xxi
ABSTRACT	xxi

RESUME

Ce mémoire présente la modernisation du système applicatif du Service Médical Interentreprises (SMIE) à travers la mise en place d'une chaîne d'intégration continue (CI) et de déploiement continu (CD) automatisée. Cette approche repose sur GitHub Actions, Docker et un environnement VPS, permettant d'automatiser les processus de développement, de test et de mise en production. Elle assure également une meilleure isolation des services, une standardisation des environnements et une réduction des erreurs liées aux interventions manuelles.

Les résultats obtenus démontrent que l'adoption des pratiques DevOps améliore significativement la qualité logicielle, la stabilité et la scalabilité du système, tout en optimisant le cycle de vie des applications. Ce travail met en lumière le rôle stratégique de l'automatisation et de la containerisation dans la transformation numérique, et offre des perspectives concrètes pour de futures évolutions technologiques au sein du SMIE.

Mots-clés : DevOps, CI/CD, Docker, GitHub Actions, VPS, automatisation, modernisation des systèmes d'information, containerisation.

ABSTRACT

This study focuses on the modernization of the Service Médical Interentreprises (SMIE) application system through the implementation of an automated Continuous Integration (CI) and Continuous Deployment (CD) pipeline. The solution leverages GitHub Actions, Docker, and a VPS environment, enabling the automation of development, testing, and production processes, while ensuring service isolation, environment standardization, and reduction of manual errors.

The results show that adopting DevOps practices significantly improves software quality, system stability, and scalability, while optimizing the application lifecycle. This work highlights the strategic role of automation and containerization in digital transformation and provides concrete insights for future technological evolutions within SMIE.

Keywords: DevOps, CI/CD, Docker, GitHub Actions, VPS, automation, information systems modernization, containerization.